

NDNSF: A DATA-CENTRIC SERVICE FRAMEWORK FOR SECURE, COLLABORATIVE,
AND DYNAMIC NETWORKED APPLICATIONS

by

Tianxing Ma

A Dissertation Proposal

Submitted for Committee Review

Academic Advisor:

Dr. Lan Wang

Major: Computer Science

The University of Memphis

September 2026

Abstract

Reading copy: black text is the revised proposal; [gold text in brackets] preserves earlier wording. Short edits appear inline; rewritten passages have an Old note. Earlier material in this topic is related text without a direct paragraph match, not necessarily a deletion. Old citations retain Origin numbering. Original illustrations can be checked at the cited Origin PDF page; revised figures and tables are retained. AI-assisted formatting only; source claims are not changed.

Networked applications need mechanisms for service discovery, invocation authorization, and application-data exchange. Changes in service availability, resources, or connectivity make these tasks more challenging. Named Data Networking (NDN) provides data-centric communication primitives, but applications still need service-level mechanisms. This dissertation investigates these mechanisms through the Named Data Networking Service Framework (NDNSF) and its extension to multi-Provider collaboration.

[Old; Origin PDF p. 2: Modern networked applications increasingly run as collections of services across mobile devices, edge nodes, cloud resources, storage systems, and users. Existing host-centric frameworks provide useful tools, but their endpoint and channel model leaves service discovery, provider selection, channel security, caching, storage, and application state in separate systems. The problem is the mismatch between this model and applications that care about named services, named data objects, and named results rather than fixed endpoint connections.]

Named Data Networking (NDN) offers named and secured data retrieval, in-network caching, multicast forwarding, and data-centric validation. These primitives match service-oriented applications in dynamic networks, but they still need to be organized into higher-level service functions: invocation, authorization, provider selection, provider collaboration, and large-data exchange.]

[Earlier material in this topic; Origin PDF p. 2: The proposal organizes the remaining framework problem into four layers: invocation, authorization, coordination, and data handling. This structure separates facts about NDN primitives from the higher-level service mechanisms that NDNSF must provide.]

NDNSF provides reusable service interfaces and Request, acknowledgment (ACK), Selection, and Response Data exchanges. Its initial prototype selects one Provider per invocation: before selection, the Request carries a discovery descriptor readable only by Providers authorized for the requested service; after selection, request-scoped keys deliver the application input only to the selected Provider. NDNSF uses attribute-based encryption (ABE) to protect the discovery descriptor from parties lacking the required decryption capability. The User need not enumerate Provider identities, but authorized candidates can read the descriptor even if they are not selected. Ser-

vice authorization combines signature validation under trust rules with current service-permission checks. Successful matching of the protected RequestNonce and ProviderChallenge values provides evidence that the parties recovered the values required for the current exchange, while request-state checks reject replayed or stale invocations. Permissions remain separate from identity-signing credentials; runtime grants and withdrawals require validated status and key updates. After selection, request-scoped keys protect application input and results for the intended participants.

[Old; Origin PDF p. 2: This dissertation proposes the Named Data Networking Service Framework (NDNSF), a data-centric service framework for secure, collaborative, and dynamic networked applications. NDNSF v0.1 is the completed foundation: it provides a generic service API, NDN-based request/ACK/selection/response messages, selective provider choice, admission metadata, name-based access control, and MiniNDN evaluation against NSC and gRPC.]

The proposed collaboration extension assigns roles to Providers whose positive ACKs pass validation and eligibility checks. Each Provider derives its input-validation rules from the signed plan. The plan binds assigned Providers to service roles and declared dependency edges. These checks constrain which intermediate Data may advance execution, including Data retrieved from caches. The study also examines known-Provider invocation, trusted local composition, streaming, and retrieval of large objects by name.

[Old; Origin PDF p. 2: The remaining dissertation work extends this foundation toward collaboration among service providers such as drones, edge nodes, storage services, and inference accelerators. The planned framework mechanisms include targeted invocation for known providers, trusted local service composition inside one container, and large-data references for artifacts that should be retrieved by name instead of carried as service payloads. These mechanisms are intended to be reusable framework features rather than application-specific shortcuts.]

Two applications provide validation settings: the unmanned aerial vehicle (UAV) application covers control, telemetry, video, and multi-view analysis; the distributed-inference application covers model retrieval, single-Provider inference, and collaborative execution of model partitions. Preliminary experiments report configuration-specific invocation latency and the number of executions observed under each configuration; they do not yet validate the collaboration extension. Planned evaluation compares complete authorization alternatives, tests dependency enforcement and recovery, and measures completion, latency, communication, computation, and credential-management costs. The expected contribution is an evaluated framework and evidence identifying

when its mechanisms benefit dynamic applications and when their overhead limits those benefits.

Keywords: Named Data Networking, service authorization, provider discovery, collaboration, distributed inference

[Old; Origin PDF p. 3: Two validation workloads drive and evaluate the framework. The UAV workload tests NDNF under mobility, intermittent connectivity, command-sensitive services, video and telemetry data, and multi-UAV collaboration. The distributed AI inference workload tests NDNF under model retrieval, dependency execution, multi-provider cooperation, and large intermediate objects.

The central hypothesis is that combining NDN's data-centric communication model with explicit service-level abstractions can reduce the complexity of building secure and collaborative networked applications. The dissertation contribution is the design, implementation, and evaluation of NDNF; the UAV and distributed inference systems validate the framework and feed requirements back into it. Keywords: Named Data Networking, service-oriented applications, data-centric security, UAV networks, distributed AI inference, provider collaboration]

Contents

1	Introduction	1
1.1	Application Motivation and Service-Level Decisions	1
1.2	The Initial NDNSF Foundation	2
1.3	Application Needs and Framework Scope	3
1.3.1	Research Gap and Design Focus	4
1.4	Extending Invocation to Multi-Provider Collaboration	7
1.5	Thesis and Research Questions	7
1.6	Proposal Scope and Application Responsibility	9
1.7	Expected Contributions and Current Foundation	10
2	Background and Related Work	12
2.1	NDN Background and Naming Responsibilities	12
2.1.1	Named Retrieval, Forwarding State, and Object Identity	12
2.1.2	From NDN Building Blocks to Service Decisions	14
2.2	Remote Procedure Call (RPC), Robotics Middleware, and Messaging Alternatives . .	16
2.2.1	RPC Interfaces and Microservice Integration	16
2.2.2	Robot Components and Asynchronous Messaging	18
2.2.3	Microservices and Edge Computing	20
2.3	Named Computation, Invocation, and Composition	20
2.4	Comparing Authorization Designs under the Same Requirements	24
2.4.1	DNMP as a Concrete Trust-Schema Design	24
2.4.2	Adapting the Alternative to NDNSF	25
2.5	Protected Discovery and Permission Management	26

3	NDNSF Framework and Proposed Extensions	28
3.1	Components and Application Responsibilities	28
3.2	Participants, Trust, and Exclusions	31
3.3	Four Named Message Types	33
3.4	Authorization and Confidentiality	35
3.4.1	Rationale for ABE-Backed Service Authorization	35
3.4.2	Authorization Mechanisms, Lifecycle, and Costs	38
3.5	Candidate Information, Selection, and Admission	41
3.6	Invocation Modes and Execution Lifecycle	42
3.6.1	Discovery, Known Providers, and Local Composition	42
3.6.2	Admission, Progress, and Failure	44
3.7	Control Messages, Streams, and Complete Objects	45
3.8	Proposed Extension: From ACKs to a Collaboration Plan	47
3.9	Proposed Dependency Admission and Execution State	48
3.10	Conditional Correctness Argument	50
4	UAV Application over NDNSF	51
4.1	Application Structure and Service Workflows	51
4.2	Operator Workflow and State Interpretation	53
4.3	Live Video, Recording, and Perception Services	54
4.4	Flight Commands and Local Safety	56
4.5	Mission Assignment and Partial Failure	57
4.6	Representative Collaboration: Multi-View Collection and Analysis	58
5	Distributed Inference over NDNSF	60
5.1	Application Structure and Inference Workflows	60
5.2	Representative Collaboration: Multi-Provider Inference	62
5.3	From a Model Graph to Runtime Assignments	63

5.4 Planning Inputs, Feasibility, and Policy Comparisons	65
5.5 Tensor Exchange and Autoregressive State	66
6 Research and Evaluation Plan	68
6.1 Cross-Application Validation	68
6.2 Preliminary Evidence and Its Limits	71
6.3 Evidence Needed for Each Research Question	74
6.4 Application Experiments and Interpretation	75
6.5 Primary Research Scope and Feasibility	81
6.6 Existing Foundation and Remaining Research Work	81
6.7 Timeline and Decision Points	83
6.8 Risks, Mitigation, and Limits	84
7 Conclusion	86
A Preliminary Prototype Observations and Evidence Limits	88
References	90

Chapter 1

Introduction

1.1 Application Motivation and Service-Level Decisions

Networked applications combine services with different lifetimes and resource needs. A UAV operator issues commands to a particular vehicle, follows changing telemetry, and retrieves video or recorded images. An inference application selects compute resources, obtains model artifacts, and exchanges intermediate results. Across these workflows, developers must associate requests, permissions, execution state, and data objects even when connectivity or available resources change.

[Old; Origin PDF p. 6: Modern networked applications are increasingly organized as collections of services rather than as single monolithic programs. A mobile robot may expose telemetry, perception, and flight-control services. A ground station may provide object detection, mapping, mission planning, and storage services. Edge devices may request AI inference services from nearby accelerators or cloud nodes. In each case, the application developer wants to invoke named services and retrieve named results without manually binding every operation to a fixed host address, a long-lived channel, and a single provider.]

[Earlier material in this topic; Origin PDF p. 6: 1.1 Motivation]

A requester may know the required service without knowing which authorized Providers are currently reachable, capable, and willing to execute it. Other operations must retain an already selected Provider, such as stopping a particular camera. The framework must therefore support runtime selection while keeping later commands and results associated with the intended Provider and operation. Here, User, Provider, and Controller are logical protocol roles: a User requests a service, a Provider offers or executes it, and a Controller administers service permissions; one node may hold more than one role. The research question is how a reusable service framework can support these decisions and data exchanges without requiring each application to define its own invocation protocol.

Named Data Networking (NDN) supports name-based Data retrieval and data-centric secu-

rity [1, 2]. Names identify Data, guide Interest forwarding, and participate in trust rules; applications can use a trust schema to constrain acceptable signing keys and guide Data validation [3]. Sync supports discovery of dataset changes so applications can retrieve available Data [4, 5]. NDNSF uses these primitives to discover Providers, authorize invocations, and track service execution. The detailed networking background is developed in Chapter 2.

[Old; Origin PDF p. 6, 7: Named Data Networking (NDN) offers a different foundation. NDN names data directly, secures Data packets directly, and decouples retrieval from fixed endpoints. These properties naturally match service-oriented applications in which users care about a named service, a named model artifact, a named telemetry object, or a named video segment rather than a particular IP connection. However, NDN by itself is a networking architecture, not a complete service framework. Application developers still need a runtime that defines how to publish service requests, authorize users and providers, select one or more providers, protect service data, and support higher-level collaboration.]

Even when one Provider suffices, candidate discovery, authorization, selection, and result retrieval remain application responsibilities. NFN provides a named-function and cached-computation model; NFaaS means Named Function as a Service; RICE means Remote Method Invocation in Information-Centric Networking; and NSC means Named Service Calls [6–9]. The Distributed Network Measurement Protocol (DNMP) provides a concrete example of Trust-Schema-based command authorization [10]. These NDN systems provide the computation, invocation, and authorization precedents against which this proposal examines NDNSF’s design choices.

1.2 The Initial NDNSF Foundation

The initial NDNSF prototype selects one Provider from multiple candidates for each invocation. The User publishes Request Data, collects acknowledgment (ACK) Data, and publishes Selection Data for the chosen Provider, which executes and publishes Response Data. Candidates offer alternatives for the same invocation; the initial prototype does not assign them to different, interdependent roles.

[Earlier material in this topic; Origin PDF p. 13: 1.5.1 Expected Answer Shape]

Discovery and authorization also matter when only one Provider executes a request. A discovery descriptor supplies information needed to assess feasibility, such as deadlines and resource

requirements; application input supplies the data needed to perform the task. In the design under evaluation, attribute-based encryption (ABE) limits descriptor access to holders of matching decryption keys. Participants check service permissions using the protected challenge values defined below (RequestNonce and ProviderChallenge), signature validation under trust rules, current permission status, and request state. Authorized but unselected candidates can read descriptors. The study also evaluates input/result protection, permission grants and withdrawals, and management costs.

1.3 Application Needs and Framework Scope

A networked application often needs more than the location of a service. A drone operator may require a camera near an area before a deadline, while an inference requester may require a particular model, sufficient accelerator memory, and an acceptable preparation delay. A service advertisement identifies a possible source of execution, but it does not establish that the source is willing and able to undertake this task now. NDNSF separates permission to provide a service, current willingness to participate, selection for an invocation, and completion of the requested work.

These distinctions also matter within an application that usually uses one device. After choosing a drone, an operator expects subsequent camera commands to affect that drone. Substituting another available Provider would change the meaning of the command. Conversely, an image-analysis request may permit any authorized compute Provider that meets its requirements. A reusable framework must support both situations without requiring the application to implement an unrelated communication protocol for each.

The discovery descriptor contains the information needed for a participation decision. For a sensing task, this may include a coarse area, a time window, and a sensor requirement. For inference, it may include model identity, expected input size, and a deadline. Full images, detailed flight instructions, and inference prompts can be delivered after selection. The boundary follows the decision: information indispensable to assessing feasibility cannot simply be withheld until after that assessment. An application should nevertheless disclose only the necessary precision. For

example, an authorized camera Provider may need an approximate search area, while an unselected candidate need not obtain the full mission route. Descriptor protection limits disclosure to authorized service Providers; it does not prevent them from learning or sharing what they legitimately decrypt.

NDNSF-UAV and NDNSF-DI expose complementary demands. The former combines device-specific operations, changing observations, live data, and physical actions. The latter combines compute selection, model preparation, large artifacts, intermediate tensors, and incremental outputs. Both require authorization and bounded request lifetimes. Their differences test whether a common invocation and named-object interface remains useful beyond one workload. Reusability is an engineering objective to be evaluated through shared mechanisms and application-specific code, rather than assumed from implementing two demonstrations.

[Old; Origin PDF p. 15, 16: These applications also contribute requirements back to NDNSF. The UAV workload stresses low-latency command invocation, structured command status, service-container composition, and large-data video references, while the DistributedInference workload stresses dependency-driven collaboration, artifact references, provider role assignment, and long-running selection diagnosis. Thus the application work has two roles in the dissertation: it is valuable in its own domains, and it is the primary method for validating and refining provider collaboration as the central NDNSF extension.]

We derive the scope by following a request from candidate discovery to execution and closure. At each stage, we identify the relationship the application needs and the failure that follows if it is not enforced.

[Old; Origin PDF p. 11, 12: This proposal structures the gap into four layers. At the invocation layer, a user needs to publish a service request, collect ACK metadata, and select one or more providers. At the authorization layer, users and providers need service-level permissions, name-based attributes, replay protection, and verifiable responses. At the coordination layer, multiple providers may need to advertise readiness, cooperate on one task, or compose local and remote services. At the data layer, services need to attach large objects such as video, model files, logs, and intermediate tensors by name rather than carrying them inside small request payloads.]

Authorization, confidentiality, and timeliness apply across stages. Permission changes can affect an invocation throughout its lifetime and are listed separately as a cross-cutting concern.

1.3.1 Research Gap and Design Focus

[Old; Origin PDF p. 11: 1.3 Research Gap]

NDN provides named Data retrieval and signature validation, but a Data name and a valid signature do not by themselves record or validate the service decision governing an invocation. The gap examined here is how a runtime service decision is carried into the admission and use of named Data.

[Old; Origin PDF p. 11: The research gap is therefore between NDN communication primitives and service-oriented application semantics. NDN provides the right lower-level building blocks: named Data, signatures, caching, forwarding state, segmented retrieval, and Sync-based discovery. A service framework must organize these building blocks into a consistent higher-level runtime.]

Candidate discovery lets Providers report eligibility and willingness, but it does not commit an executor. A Provider's permission to offer a service authorizes that capability; it does not assign the Provider to a particular invocation or workflow role. Selection is the binding step: the initial prototype commits one Provider per invocation, while the collaboration extension commits Provider-to-role assignments and dependency edges.

[Old; Origin PDF p. 6: Today's service frameworks provide valuable engineering tools, but TCP/IP communication is still centered on reaching endpoints and maintaining channels. Remote procedure call (RPC) systems, REST services, and microservice platforms typically identify service endpoints by host addresses, ports, or channel endpoints. Robotics middleware provides publish-subscribe or request-response communication, but often assumes an operational domain where identity, trust, network reachability, and service discovery are configured outside the service invocation itself. The problem is not that these tools are unusable; the problem is that mobile and data-heavy applications want to name the service and the data objects involved in the workflow, while the networking substrate mainly exposes where an endpoint is and how to keep a path to it. When applications become mobile, intermittent, multi-provider, and security-sensitive, developers must rebuild mechanisms for discovery, retry, authorization, integrity verification, data retrieval, and provider selection.]

For a dependent task, signature verification authenticates the producer but does not establish that the object was produced by the assigned predecessor for the active invocation or satisfies the declared dependency. The consumer must therefore check the producer, exact object name, dependency edge, and execution context.

Prior invocation and command-authorization systems address parts of this problem. NDNSF investigates a reusable protocol connecting these decisions, with authorization spanning invocation and data retrieval.

[Earlier material in this topic; Origin PDF p. 12: NDNSF addresses this layered gap by treating service invocation, authorization, provider selection, collaboration, and large-data reference handling as reusable framework mechanisms rather than application-specific glue code.]

To make this gap operational, Table 1 maps each invocation stage to the relationship the framework must preserve, the failure caused by its absence, and the validation planned in Chapter 6.

[Old; Origin PDF p. 7: This gap also emerged from the application-driven development in this dissertation. Building the UAV application exposed the need for stable service selection, command authorization, telemetry freshness, local service composition, and large video/recording object handling. Building the distributed inference workload exposed the need to coordinate artifact provisioning, tensor dependencies, provider roles, and large intermediate data. These experiences motivate treating the gap as a framework problem rather than as isolated application glue code.]

This table is a requirements-to-evidence map; it does not claim that all proposed

Table 1: From invocation requirements to failure cases and proposed validation.

Stage or concern	Required relationship	Failure if missing	Proposed checks
Candidate discovery	Eligible candidates report willingness and feasibility; descriptor access is restricted	Ineligible candidate selected or descriptor disclosed to unauthorized readers	ACK eligibility and descriptor access
Selection and start	The intended Provider starts only a permitted, active invocation	Wrong device, unauthorized work, or repeated start	Permission, Selection, and replay checks
Input and intermediate Data	Correct producer, object or stream, and permitted dependency	Wrong, stale, or incomplete input advances execution	Object/stream validation; role, edge, and cache tests
Closure or replacement	Results match the active operation; closed work cannot advance its replacement	Late results revive closed work or complete the wrong operation	Deadline, cancellation, and replacement tests
Permission changes	Permission updates take effect after status validation and required key updates	A grant is unusable or withdrawn permission remains accepted	Grant/withdrawal propagation and rejection delay

collaboration checks have already been implemented or validated.

The evaluation asks whether its checks enforce the intended relationships and whether the resulting cost is justified for the studied workloads.

[Earlier material in this topic; Origin PDF p. 15: The core dissertation question is therefore not whether two applications can be built, but whether NDNSF can provide reusable data-centric service mechanisms that remain valid across two very different workloads.]

For each workflow in Chapters 4 and 5, validation will identify the applicable rows, test their

failure cases through the corresponding framework paths, and record the outcomes. Single-Provider workflows do not require collaboration-role checks.

Chapter 6 specifies the proposed tests. Coverage is limited to the declared workflows; the exclusions below define the remaining scope boundaries.

1.4 Extending Invocation to Multi-Provider Collaboration

Some tasks require complementary services from different Providers. Independently invoking each service does not by itself specify which Provider fills each role or which intermediate objects a dependent service may accept. This proposal investigates an extension that makes those assignments and dependencies explicit while retaining the invocation foundation.

For example, nearby unmanned aerial vehicles (UAVs) could collect different views of a ground area for analysis by a compute Provider. The extension would assign sensing and processing roles. The compute Provider would check that each image comes from the UAV assigned to the required input. UAVs may need area and deadline information before deciding whether to return a positive ACK. After Selection authorizes the sensing tasks, the UAVs produce images for the assigned compute Provider. Selection supplies execution details and the required access information, not images that have yet to be captured. Descriptor encryption restricts who can read the descriptor; it does not hide names or traffic patterns. This case motivates the extension; recognition algorithms remain an application choice.

1.5 Thesis and Research Questions

[Old; Origin PDF p. 13: 1.5 Research Questions]

The central hypothesis is that a common named-Data protocol can separate candidate discovery from authorized execution and make runtime role assignments enforceable when intermediate objects are used. This would let UAV and inference applications share invocation and dependency-validation mechanisms while retaining their own task semantics. The study tests this hypothesis through message/state invariants, cross-application use of the same mechanisms, and

matched measurements of completion and cost. It evaluates single-Provider invocation and multi-Provider collaboration separately. RQ1. Under what numbers of users and services, degrees of permission overlap, membership-change rates, and withdrawal frequencies does ABE-backed authorization provide a favorable confidentiality–management-cost tradeoff relative to a DNMP-inspired role-constrained authorization alternative for protected Provider discovery and invocation? RQ2. How can an ACK-based, single-Provider invocation be extended so that, after candidate collection closes, runtime-selected Providers receive explicit workflow roles and named dependency scopes, and accept intermediate Data—whether retrieved from a producer or a cache—only when it matches the committed producer, name, dependency edge, and execution context? RQ3. Under what workload, network, and failure conditions do ABE-backed discovery, role/dependency admission, and bounded recovery improve completion and latency, and when do their communication, computation, and credential-management costs outweigh the benefits?

[Earlier material in this topic; Origin PDF p. 12, 13: • UAV network applications represent mobile, command-sensitive, real-time service-container workloads, where intermittent connectivity, command authorization, telemetry, live video, recording, object detection, and multi-UAV missions must be coordinated across a ground station and multiple drones.

• Distributed AI inference represents artifact-heavy, multi-stage compute/data collaboration workloads, where model shards, runtime artifacts, intermediate tensors, large-data objects, and heterogeneous inference providers must be selected, verified, and coordinated.]

RQ1 evaluates the invocation foundation, including a DNMP-inspired role-authorization alternative paired with content encryption and adapted to the NDNSF workflow. RQ2 investigates the collaboration extension through UAV and distributed-inference dependencies. RQ3 evaluates the utility and cost of the mechanisms studied in RQ1 and RQ2, rather than introducing another protocol; invocation and collaboration are measured separately.

[Old; Origin PDF p. 12, 13: This dissertation argues that dynamic service-oriented applications expose a recurring mismatch between TCP/IP’s connection-oriented model and service-oriented applications’ data-centric needs: applications want to operate on named services, named data, and verifiable results, while the network stack usually exposes addresses, channels, and endpoints. A data-centric service framework can close this gap by making service invocation, authorization, provider selection, collaboration, and large-object retrieval explicit framework-level abstractions over NDN names and verifiable Data packets.

The dissertation is organized around the following research questions. RQ1. What problems in host-centric service frameworks can be addressed by data-centric service frameworks? RQ2. How should NDNSF expose service invocation, authorization, provider selection, collaboration,

and large-object retrieval as generic framework abstractions without forcing each application to design a custom NDN protocol? RQ3. What reliability, security, and latency tradeoffs arise when NDN service requests are published to multiple providers and resolved through ACK metadata, provider selection, and collaboration, and how should supporting mechanisms such as Targeted invocation and compensation be used? RQ4. How can UAV network applications evaluate NDN under mobile, command-sensitive, real-time service-container workloads? RQ5. How can distributed AI inference evaluate NDN under artifact-heavy, multi-stage compute/data collaboration workloads?]

[Earlier material in this topic; Origin PDF p. 12, 13, 14: 1.4 Thesis Statement]

The dissertation is expected to answer these questions in the following form. RQ1 should identify a set of recurring service-application problems that are not merely implementation gaps in existing systems, but consequences of binding services to hosts, sessions, or channels. RQ2 should show that NDN can expose service invocation, authorization, provider selection, and provider collaboration as reusable framework mechanisms, with local composition and large-data references serving as supporting abstractions rather than application-specific NDN protocols. RQ3 should characterize when one-to-many request publication, ACK metadata, provider selection, and provider collaboration improve reliability, and when supporting mechanisms such as Targeted invocation or compensation are needed to control latency and recovery costs without weakening security checks. RQ4 should show that UAV workloads exercise the same framework mechanisms under mobility, command sensitivity, real-time video, and multi-drone collaboration. RQ5 should show that distributed inference exercises the same mechanisms under artifact provisioning, tensor dependency exchange, provider role assignment, and long-running distributed execution.]

1.6 Proposal Scope and Application Responsibility

[Old; Origin PDF p. 15: 1.7 Proposal Scope Boundary]

The common framework scope includes discovery, permission checks, request correlation, selection, named-object exchange, and execution lifecycle. Applications define service meaning, feasibility constraints, model or sensor compatibility, and acceptable outcomes. The collaboration extension adds task roles and dependencies to that common scope. Known-Provider invocation, trusted local composition, and large-object support also matter for single-Provider tasks and are evaluated where their use is justified.

[Earlier material in this topic; Origin PDF p. 15: For clarity, this proposal treats NDN v0.1 as a completed core contribution of the dissertation, not as the final complete system. NDN v0.1 establishes the basic architecture and feasibility of a data-centric service framework. The remaining dissertation work extends this foundation into an application-validated framework. The main framework extension is provider collaboration: how multiple authorized providers discover requests, advertise capabilities, are selected, exchange state or artifacts, and jointly complete a service task. Targeted invocation, trusted local invocation, and large-data references are treated as supporting

mechanisms that make this collaborative service model practical under low-latency, same-process composition, and large-object workloads. These mechanisms are not UAV-only or AI-only features; they are proposed NDN-SF framework mechanisms whose usefulness will be evaluated through two demanding application classes.

The UAV application is used to validate NDN-SF as a service-container framework for mobile, command-sensitive, real-time systems. Its design exercises typed telemetry, mission, video, and safety information; Targeted flight-control commands; adaptive live video; encrypted local recording; object-detection service composition; and multi-drone patrol workflows. The DistributedInference application is used to validate NDN-SF as a framework for artifact-heavy, multi-stage compute/data collaboration. Its design exercises automatic model plan generation, reference-based model and activation artifacts, predictable prefetchable object names, and dependency-driven execution over tensor DAGs. The dissertation therefore does not present UAV and DistributedInference as isolated demos; it uses them as two complementary stress tests for the same data-centric service framework thesis.]

Repository support retains named objects when a workload needs persistence. Storage-system design, arbitrary workflow optimization, hard real-time flight control, safety certification, and unrestricted model partitioning are outside the research scope. Reuse will be examined by tracing the shared framework components and application-specific adapters in each workload. The two applications thus provide contrasting cases for evaluating a common service abstraction.

1.7 Expected Contributions and Current Foundation

[Old; Origin PDF p. 14: 1.6 Expected Contributions]

The intended contribution is an evaluated service framework that combines discovery, authorized invocation, named-object exchange, and execution management, with an extension for complementary Provider roles. The research will determine when its authorization choices are worthwhile, how runtime assignments constrain intermediate inputs and replacement executions, and which mechanisms can be shared across UAV and inference applications. Matched workload measurements will establish benefits and limits, including conditions in which the mechanisms offer no advantage. The contribution concerns these service-level abstractions and their evaluation, not new NDN or encryption primitives.

[Old; Origin PDF p. 14: The expected dissertation contributions are:

1. A review and gap analysis of service-oriented application frameworks with a focus on why dynamic, secure, collaborative applications still require a new data-centric framework.

2. The design and implementation of NDNFSF as a generic dynamic C++ service framework over NDN.

3. Extensions to NDNFSF centered on provider collaboration, with supporting mechanisms for targeted invocation, trusted local service composition, and large-data references.

4. A UAV service-container application over NDNFSF, including flight control, telemetry, video, encrypted recording, object detection, and collaborative patrol services.

5. A distributed AI inference application over NDNFSF, including model splitting, service plans, reference-based artifact exchange, and automatic collaboration-plan generation.

6. An evaluation of NDNFSF and the application prototypes using MiniNDN, SITL-based UAV experiments, and selected real-machine validation.]

The existing NDNFSF implementation provides an invocation prototype and initial collaboration support. Its current planning path closes candidate collection before validating a plan-commit state (defined in Chapter 3), while preliminary invocation campaigns provide baseline evidence. Building on this foundation, the study will evaluate role and dependency checks within a request, end-to-end correctness, and application benefits. Chapter 6 presents the preliminary findings and planned experiments.

Chapter 2 establishes the background and alternatives. Chapter 3 presents the framework and proposed extension. Chapters 4 and 5 describe the UAV and distributed-inference applications. Chapter 6 brings together preliminary evidence, proposed experiments, remaining work, and the timeline; Chapter 7 concludes the proposal.

[Earlier material in this topic; Origin PDF p. 12: NDNFSF v0.1 is a completed core contribution of this dissertation and demonstrates the feasibility of this direction. The remaining dissertation work extends this foundation into a framework for real dynamic service applications and evaluates it through two high-pressure domains with different service requirements:]

Chapter 2

Background and Related Work

[Earlier material in this topic; Origin PDF p. 17: Frameworks for Service-Oriented Applications]

2.1 NDN Background and Naming Responsibilities

[Old; Origin PDF p. 7: 1.2 NDN Building Blocks for NDN-SF]

2.1.1 Named Retrieval, Forwarding State, and Object Identity

NDN retrieves Data by name using Interest and Data packets [1, 2]. An application expresses an Interest describing the Data it seeks. A matching Data packet can be supplied by its producer or an intermediate cache. Forwarders use forwarding information to direct Interests and pending-Interest state to deliver returning Data. This permits network-layer reuse of named Data without requiring the consumer to identify its present storage location. Availability still depends on forwarding, connectivity, and a reachable copy; a name is not a delivery guarantee.

[Old; Origin PDF p. 7: This proposal assumes several NDN concepts, but they are used here for a specific purpose: to explain why NDN-SF is a data-centric service framework. NDN is data-driven: a consumer sends an Interest packet for a named object, and the network returns a matching Data packet from a producer, cache, or other available source [1]. This basic exchange is not yet a complete service invocation, but it supplies the communication primitive that NDN-SF builds on: the application names what it wants, not the host that should send it; the returned Data packet is independently named and signed; and retrieval can succeed even when the original producer is not on the same path as the consumer.]

[Earlier material in this topic; Origin PDF p. 8: NDN is data-centric in its security model as well as in its naming and forwarding model. Each Data packet carries a producer signature, allowing consumers to verify integrity and producer authenticity regardless of whether the Data came from the original producer, an in-network cache, or a repository [1]. This property is central to NDN-SF: service requests, ACKs, selections, responses, permissions, and large artifacts can all be treated as named and verifiable objects. However, signatures alone do not decide who is allowed to call a service, which provider is authorized to offer it, or which attributes should protect each service message. Those higher-level decisions are framework responsibilities.]

Names have several related uses in this proposal. A service name identifies the requested operation. A message name, together with the message's validated fields, identifies an invocation exchange. An object name identifies data such as an image, model fragment, or output tensor. Names also participate in trust rules that constrain acceptable signing keys [3]. These uses should agree, but should not be conflated: matching a service prefix does not by itself prove service permission, and a valid producer signature does not prove that the object is the input assigned to the current execution.

[Old; Origin PDF p. 7: NDN names have three roles that are important for this dissertation. First, they identify application data, such as telemetry, video segments, model artifacts, service requests, ACKs, selections, and responses. Second, they are part of the security model: trust schemas and access-control attributes can be defined over name prefixes. Third, they guide routing and forwarding because Interests are forwarded by name prefix rather than by destination address. NDNSF uses the same naming foundation for service-level objects, so a service workflow can refer to both actions and large data by name.]

For example, an illustrative image name could have the form `/org/uav7/view/task42/view2/version1` under the sensing UAV's producer namespace. The application must define what the task, view, and version components mean and validate the signer. A processing Provider can refer to that exact image rather than rename it as its own observation. This example explains the naming principle, not a mandatory NDNSF wire format. Names and traffic patterns can remain visible even when Content is encrypted. Sensitive identifiers should therefore not be embedded in public names merely because the associated payload is protected.

[Earlier material in this topic; Origin PDF p. 10: For access control, NDNSF builds on NAC-ABE, which combines NDN naming with attribute-based encryption so that service-level authorization can be expressed over names rather than transport connections [7]. NDNSF uses this capability together with controller-issued permissions, service-level attributes, and per-invocation tokens. The result is that NDN supplies the data-centric communication substrate, while NDNSF supplies the service-layer runtime: who may call a service, who may provide it, how providers are selected, how messages are protected, and how large named objects are attached to service workflows.]

The forwarding Content Store offers opportunistic caching. A repository provides an application-managed place to retain objects for later retrieval. Neither a cache hit nor a successful digest check supplies the complete admission decision for a dependent service. The consumer still needs the producer's authority, the intended object identity, and the current invocation or col-

laboration association. Likewise, a Data packet's FreshnessPeriod is NDN metadata that guides cache-freshness treatment; it is not a service deadline or a substitute for message expiry, current permission checks, or application replay state.

Sync helps participants learn about changes in a named dataset [4, 5]. Applications can then retrieve and validate the announced Data. Thus, publication in NDN means making a named object available and announcing it through the configured dissemination mechanism. It should not be read as an unsolicited push of a Data packet through arbitrary forwarders. Sync recovery can repair missing announcements or trigger retrieval while time remains. It cannot ensure completion after an application deadline or during an indefinitely disconnected interval.

These distinctions define the meaning of data-driven execution here. A validated Request, ACK, Selection, result, or declared intermediate object supplies evidence for a local state transition. Timers and local scheduling policies still exist. A timeout can close collection or terminate an operation, and a planning algorithm can decide which assignment to publish. Decisions that affect other participants are communicated through named, verifiable messages; a local callback alone cannot inform a remote participant. This does not imply that all runtime state is network Data or that packet arrival alone authorizes execution.

2.1.2 From NDN Building Blocks to Service Decisions

The Forwarding Information Base (FIB) supplies name-prefix forwarding information, the Pending Interest Table (PIT) records pending retrievals, and the Content Store (CS) holds cached Data [11]. Compatible pending Interests can share a returning Data packet. This network-level aggregation is distinct from selecting several service executors. A PIT entry also has a different lifetime from a service operation: its expiration does not cancel an application handler or erase the User's request state.

[Old; Origin PDF p. 8: NDN forwarders maintain three data structures that are important for NDN's design. The Forwarding Information Base (FIB) forwards Interests by name prefix. The Pending Interest Table (PIT) records unsatisfied Interests and aggregates multiple Interests for the same name, allowing one returned Data packet to satisfy multiple consumers. The Content Store (CS) caches Data packets for future retrieval. Together, these structures support stateful forward-

ing, in-network caching, multicast-style delivery, and adaptive retrieval under mobility or failures [2]. For service applications, this means that service-related data can be named and retrieved independently of a fixed address or connection. Service invocation semantics, however, still have to be built above these packet and forwarding primitives.]

Table 2: NDN building blocks and the service decisions that remain above them.

Building block	NDNSF use	Additional service responsibility
Interest/Data and names	Retrieve messages, results, and object segments	Define the operation, request identity, and completion condition
FIB, PIT, and CS	Forward and reuse named Data	Select work, track its lifecycle, and check acceptable object versions
Signatures and trust schema	Validate signing authority and Data integrity	Check service permission and assignment to this invocation
Dataset synchronization (generic Sync; NDN-SVS implementation)	Discover newly published message names	Validate messages and process ACK, Selection, and timeout state
Content encryption	Restrict access to protected Content	Define recipients, distribute keys, and apply permission changes

[Old; Origin PDF p. 11: Table 1: How NDN building blocks motivate NDNSF mechanisms. NDN Building Block NDNSF Mechanism Data packets Request, ACK, Selection, and Response are service message objects carried as named Data. Interests Interests fetch named Data; NDN-SVS uses Sync Interests to announce dataset updates. Data signatures and name-based trust Service messages, permissions, and large objects can be verified independently of the path or cache that returned them. FIB/PIT/CS forwarding and caching Service-related data can be retrieved by name and remain meaningful across mobility, caching, and intermittent connectivity. NDN Sync / SVS Users and providers discover newly published service events without binding a service to a fixed endpoint. Segmented Data retrieval Large models, videos, logs, and intermediate tensors are carried by reference rather than embedded in small service payloads. NAC-ABE and named attributes Service-level permissions and confidentiality policies are expressed over names and attributes rather than transport channels.]

[Earlier material in this topic; Origin PDF p. 9: IP hourglass Applications email web voice Application protocols SMTP HTTP RTP Transport TCP UDP IP packets Links Ethernet PPP Media copper fiber radio NDN hourglass Applications browser chat sensing Data abstractions file stream service Security Named content chunks Forwarding strategy Networks/media IP UDP wireless Individual apps Every node Individual links Figure 1: Conceptual comparison of IP and NDN hourglass models, redrawn from the NDN architecture discussion [1]. Interest Packet Name Selectors order preference, publisher filter exclude filter Nonce Guiders scope, Interest lifetime Data Packet Name MetaInfo content type, freshness period Content Signature signature type, key locator signature bits Figure 2: Simplified NDN Interest and Data packet structure, redrawn from the NDN architecture paper [1].]

In the software stack, the local NDN forwarder handles packet forwarding, while ndn-cxx

supplies application packet, signing, validation, and retrieval interfaces [12]. NDN State Vector Sync (NDN-SVS) supplies a concrete dataset-synchronization implementation [13]. NDNSF organizes these facilities into service interactions. Its service policies do not require forwarders to interpret GPU load, flight readiness, or a collaboration role. Those meanings belong to the framework and application contracts.

[Old; Origin PDF p. 10: The implementation is built on the standard NDN software stack. NFD provides the local forwarder and face abstraction, while `ndn-cxx` provides the C++ application library for names, packets, faces, signing, validation, and segmented object retrieval [6]. These libraries allow NDNSF to use ordinary NDN names and packets rather than inventing a separate transport format. They also make it possible for applications to pass large artifacts by reference: models, video recordings, images, logs, and intermediate inference data can be segmented into named Data packets, signed, optionally encrypted, retrieved by name, and verified outside the small service-request payload.]

[Earlier material in this topic; Origin PDF p. 9, 10: Data-driven retrieval and in-network caching are useful, but service invocation needs more than fetching a single object. Multiple providers may need to learn that a request has been published, users may need to collect ACK metadata before selecting providers, and long-running work may need status diagnosis. NDNSF therefore uses NDN Sync, especially State Vector Sync as implemented by NDN-SVS, to let users and providers discover newly published service messages without binding the service to one fixed endpoint [3–5]. In NDNSF, Sync is not the service semantics itself; it is the dissemination substrate over which the framework publishes Request, ACK, Selection, and Response messages as named Data.]

2.2 Remote Procedure Call (RPC), Robotics Middleware, and Messaging Alternatives

2.2.1 RPC Interfaces and Microservice Integration

[Old; Origin PDF p. 17: 2.2 Host-Centric RPC and Microservice Frameworks]

Remote procedure call (RPC) frameworks provide typed interfaces for invoking operations on another process or host. gRPC is one RPC implementation that supports unary and streaming calls [14]. Robotics middleware such as ROS provides reusable communication abstractions for robot components [15, 16], while message-oriented middleware supports decoupled producers and consumers [17]. These systems are relevant alternatives to application-specific networking. NDNSF does not claim that endpoint discovery, asynchronous interaction, or distributed application composition is absent from existing middleware.

[Earlier material in this topic; Origin PDF p. 17, 18, 19, 20: 2.1 Purpose of This Review

2.2.1 What These Systems Solve

However, NDN SF targets a different layer. ROS and ROS2 are strong middleware systems for local and distributed robot components, while NDN SF focuses on named, secure, network-visible service invocation over NDN. In the proposed UAV application, the drone app can still be viewed as a service container, but external interactions such as ground-station commands, telemetry discovery, video retrieval, and cross-drone collaboration use NDN SF service names and data-centric verification. This distinction is important: NDN SF is not a replacement for flight controllers or robotics middleware, but a network service framework that can connect such components across dynamic NDN environments.

2.4 Message Queues and Event Systems

However, many message queue deployments rely on brokers or logically centralized infrastructure. Security is often enforced by broker access control and channel encryption rather than by data objects that remain verifiable wherever they are stored or forwarded. For mobile or disconnected edge environments, broker availability and placement can become a limiting assumption. Moreover, queues do not by themselves define how a user should choose among multiple providers, verify that a provider is authorized to serve a specific service, or retrieve large service artifacts as independently verifiable data.]

Endpoint resolution identifies a destination, but does not necessarily establish that the destination is authorized and currently willing to execute a particular invocation. Middleware or the application may add those policies; this proposal treats their expression and evidence as part of the comparison.

[Earlier material in this topic; Origin PDF p. 17: This chapter is not intended to be a broad survey of every service framework. Instead, it reviews what existing systems have already achieved and identifies the open issues that motivate NDN SF. The key question is not whether systems such as gRPC, ROS, microservices, or NDN Service Call are useful; they clearly are. The key question is why dynamic, secure, collaborative applications still require another framework despite the existence of those systems.]

The comparison concerns how the same application requirements are expressed. An RPC experiment must state how endpoints are resolved, connections are managed, retries are performed, and failures are detected in the selected implementation. An NDN experiment must state how names are discovered, how configured synchronization and retransmission operate, how Data is validated, and how Content Store behavior is handled. A typed call is not inherently less secure than named-object exchange; security depends on the accompanying authentication, authorization, and encryption mechanisms. Likewise, network-layer caching is useful only when object reuse is permitted by the workload and operation semantics. Reusing an image does not mean that an invocation with physical side effects may be repeated safely.

[Old; Origin PDF p. 17, 18: However, the abstraction becomes less natural for applications with mobility and intermittent connectivity. A user of a UAV video service may care about the latest frames from drone A, not whether drone A is currently reachable through a particular IP address. A distributed inference client may care about the service name for a model stage, not a fixed GPU server. A mission-planning application may need any available drone that satisfies a role, not exactly one preconfigured endpoint. Host-centric frameworks can support these scenarios, but the burden shifts to external discovery, retry, state tracking, and application-specific coordination.]

The first issue is endpoint dependence. Even if modern service meshes hide IP addresses behind logical service names, the underlying communication still depends on reaching an endpoint through host-centric routing. The second issue is data lifetime. Once a response, video chunk, model shard, or telemetry log is stored outside the original channel, applications must add their own verification and encryption model. The third issue is provider multiplicity. RPC normally selects a single server before the request is sent; NDNSF instead allows multiple providers to observe a request, decide whether to acknowledge it, and expose metadata that helps the user choose. These differences motivate a data-centric service framework rather than another IP endpoint wrapper.]

[Earlier material in this topic; Origin PDF p. 18, 19: 2.2.2 Open Issues for NDNSF]

The remaining gap is that robotics middleware is not a general data-centric network security framework. Trust anchors, certificate deployment, data-at-rest protection, multi-provider service selection, and cross-domain service authorization are not the main abstractions. In addition, a UAV application deployed across a ground station, multiple drones, and intermittent wireless links still needs naming, verification, and recovery mechanisms that are robust outside a single local robotics domain.]

For NDNSF, the relevant question is how service decisions and application data can remain linked through names and verifiable objects. The evaluation must test that link under stated workloads rather than equate a difference in networking abstraction with superiority. The named-computation and command-authorization systems discussed below provide more direct protocol precedents.

2.2.2 Robot Components and Asynchronous Messaging

Robot software separates camera, telemetry, control, and perception components, often combining service calls with continuously updated publish–subscribe topics [15, 16]. NDNSF’s service-container design adopts this separation between component behavior and communication support. It does not replace the flight controller or imply that robotics middleware cannot implement authorization. The comparison instead asks how an incoming operation identifies the intended vehicle and how stored observations remain associated with their producer and operation.

[Old; Origin PDF p. 19: Robotics middleware, including ROS and ROS2 [10, 11], provides a more application-aware model for robot components. Nodes can publish topics, call services, and interact through standardized message types. ROS2 improves quality-of-service control and deployment flexibility compared with earlier ROS systems. These frameworks are important precedents for UAV applications because they treat sensors, control, telemetry, and perception as composable modules.]

ROS is particularly important for this dissertation because it shows that robotics applications are naturally service-oriented. A UAV system is not a single communication flow. It contains flight-control commands, telemetry streams, camera frames, mapping, mission planning, object detection, logging, and health monitoring. ROS-style composition validates the idea that UAV software should be decomposed into services.]

[Earlier material in this topic; Origin PDF p. 19: 2.3 Robotics Middleware and Service Composition
2.3.1 Lessons for UAV Service Containers]

Message brokers provide asynchronous delivery and buffering for logs, telemetry, and workflow events [17]. The relevant design questions are where data is retained, who authorizes publication and consumption, and how delivery acknowledgment differs from task completion. A broker may be appropriate for a stable deployment. NDN's named retrieval provides a different placement and validation model, but still requires an explicit retention policy when late retrieval matters. Neither messaging abstraction alone chooses a willing execution Provider or defines safe repetition of a physical command.

[Earlier material in this topic; Origin PDF p. 18, 20: RPC and microservice systems solve several problems that NDN should not ignore. First, they give developers a familiar programming model: a service has an interface, a request type, a response type, error handling, and often a deadline. Second, production systems around gRPC and REST provide mature tooling for load balancing, logging, monitoring, deployment, and versioning. Third, channel security such as TLS gives a practical and widely deployed way to protect client-server communication. For stable cloud services, these features are often sufficient.]

Message queues and publish-subscribe systems decouple producers and consumers in time [12]. They are useful for telemetry, logs, asynchronous workflows, and elastic back-end processing. This is relevant to NDN because many service applications include both request-response actions and event streams. For example, a drone may receive a command as a service request while publishing telemetry or video metadata asynchronously. A distributed inference pipeline may publish intermediate results and execution events while serving user requests.]

2.2.3 *Microservices and Edge Computing*

CFEC studies computation offloading across vehicular fog, edge, and cloud resources using mobility and resource information [18]. MIA-NDN studies microservice-aware Interest aggregation and PIT lifetime management based on input characteristics [19]. SECaaS studies the deployment of security functions at the multi-access edge, including access-control enforcement and anonymization [20]. These works widen the comparison beyond function invocation: resource placement, forwarding state, and deployable security functions are relevant parts of the surrounding system. [Earlier material in this topic; Origin PDF p. 17: RPC systems such as gRPC [8], HTTP/REST systems [9], and microservice platforms have become the dominant way to build distributed software. They provide typed APIs, transport bindings, load balancing, service registration, and production tooling. These systems work very well when services can be identified by stable endpoints, when channels can be established and maintained, and when security can be expressed as channel security plus service-side authorization.]

Differences in these systems' objectives do not, by themselves, establish unique NDNF capabilities. Placement feasibility does not establish invocation permission, parameter-aware aggregation does not determine the assigned predecessor in a collaboration, and deploying an access-control service is a different question from authorizing calls to that service. Conversely, NDNF's application-layer checks do not replace these systems' resource-management or forwarding mechanisms. A claim of superiority would require a common workload and matching requirements; their published results are not interchangeable baselines for the experiments in this proposal.

[Earlier material in this topic; Origin PDF p. 18: NDNF does not attempt to replace this engineering knowledge. Instead, it asks which assumptions break when service providers are mobile, intermittently connected, replicated, or embedded in resource-constrained edge environments. In these settings, a service call is not only a channel operation. It is also a decision about which provider is currently authorized, reachable, capable, and worth selecting. Existing host-centric frameworks can implement this behavior, but the logic is typically distributed across service discovery systems, load-balancers, application retries, middleware plugins, and service-specific policy code.]

2.3 **Named Computation, Invocation, and Composition**

Named-computation and invocation systems provide direct precedents for this proposal. NFN and NFaaS study named functions, cached computation, or execution placement [6, 7]; RICE provides

an invocation handshake with authentication and security information [8]; and NSC specifies signed invocation Interests validated with an application-provided Trust Schema [9]. DNMP provides a concrete example of schema-constrained role authorization [10]. CFN and CaSCON further cover computation graphs, worker placement, or cache-assisted composition [21, 22]. Therefore, named computation, caching, workflow composition, and role-authorized calls are not claimed as isolated NDN SF novelties.

[Earlier material in this topic; Origin PDF p. 20, 21, 22, 23, 24: 2.5 NDN Computation and Service Frameworks

NDN already provides useful primitives for service-oriented systems: hierarchical names, receiver-driven retrieval, in-network caching, adaptive forwarding, and data signatures [1]. A service framework over NDN must bridge the gap between these primitives and application-level service semantics. Prior NDN work explores several regions of this design space.

2.5.1 Function-Centric Computation

Named Function Networking (NFN) treats computation as a first-class named object by embedding function expressions in names [13]. This model is powerful because it allows computation and data retrieval to be composed within the naming system itself. It also suggests a deep integration between content names and computation names, which is attractive for in-network processing.

The trade-off is complexity. Function-centric naming can make names highly expressive, but it also makes execution semantics, trust, caching, and deployment more difficult to reason about. UAV control, live video, and distributed AI inference require explicit service authorization, provider selection, long-running state, and application-defined execution policies. NDN SF therefore chooses a service-invocation abstraction rather than a fully general function-expression abstraction.

2.5.2 Serverless and Edge-Oriented NDN Frameworks

NFaaS extends NDN toward serverless computing by treating functions as deployable units that may be executed at edge locations [14]. Other edge-oriented frameworks, such as CFEC for information-centric Internet of Vehicles, explore low-latency microservice execution and dynamic placement in edge environments [15]. These systems highlight an important direction: service execution should move closer to users, data sources, or resource-rich edge nodes.

The open issue is that dynamic placement is only one part of service-oriented application support. Applications also need a stable API, authorization for both users and providers, multi-provider selection, large-object handling, and clear deployment procedures. NDN SF focuses on these runtime service semantics. It can coexist with future placement mechanisms; for example, a placement algorithm could decide which drones, storage components, or inference providers should advertise a given NDN SF service.

RICE provides remote method invocation in ICN using mechanisms such as handshakes and thanks to represent computation parameters and results [16]. NSC proposes Named Service Calls for NDN and offers an RPC-like abstraction for remote execution [17]. These systems are close to NDN SF's goals because they directly address service invocation rather than only content retrieval.

2.5.4 Application-Specific NDN Frameworks

Other NDN systems focus on particular domains. DNMP builds a secure network measurement framework using NDN synchronization and hierarchical topics [18]. MIA-NDN explores microservice-centric Interest aggregation for NDN [19]. SECaaS studies security-as-a-service in multi-access edge environments [20]. These projects demonstrate that NDN can be adapted to practical service-like workloads, especially when names and application semantics are designed together.

2.6 NDN-Based Service Systems

NDN Service Call (NSC) [17] and other NDN application frameworks show that NDN can support service invocation without IP endpoint binding. NDN’s Interest/Data exchange, name-based routing, in-network caching, and data signatures address several issues in IP-based systems. These systems are therefore important evidence that NDN is a promising foundation for services.

- generic dynamic service APIs,
- controller-authorized user and provider permissions,
- application-defined selection strategies,
- large-data references,

2.7 Design Patterns and Trade-offs

2.8 Open Issues]

The narrower research question is how to enforce a late-bound collaboration after the positive and negative ACK collection window closes. The proposed extension assigns eligible Providers to application roles and derives predecessor and dependency checks from that assignment. A valid signature proves provenance and a request identifier provides correlation, but neither alone proves that an intermediate object was produced by the Provider assigned to the required predecessor role or is permitted on the declared dependency edge. The contribution under study is this task-specific connection between runtime assignment and Named Data admission, not a new workflow or manifest format.

[Old; Origin PDF p. 22: The key limitation is that they do not provide the complete set of mechanisms needed by the application classes targeted in this dissertation. NSC is primarily one-to-one: a caller contacts an executor, and the protocol is not designed around simultaneous provider discovery, provider ACK metadata, or multi-provider selection. RICE provides stronger invocation semantics but requires changes to the forwarding plane. NDN-SF keeps the framework at the application/runtime layer and uses NDN Data publication and Sync to support one-to-many service request dissemination without modifying NFD.]

[Earlier material in this topic; Origin PDF p. 21, 22, 23, 24, 25: 2.5.3 Remote Invocation Frameworks

The limitation is generality. Application-specific frameworks often make assumptions about a particular data type, service domain, aggregation pattern, or deployment topology. NDN-SF instead aims to provide a reusable framework layer that can support different application services, including UAV control, video, object detection, distributed inference, storage, and collaboration.

The proposed UAV and DistributedInference applications are intended to test whether the framework is general enough for multiple domains rather than only one carefully tuned application.

- one-to-many request publication,
- provider ACK metadata,
- adaptive admission control,
- provider collaboration, and

The related work suggests several recurring design patterns. First, service frameworks must decouple service names from provider locations. Second, large inputs and outputs should be represented as named data objects rather than embedded directly in small request payloads. Third, service state usually cannot rely only on NDN’s Pending Interest Table because service execution may be long-running, multi-step, or collaborative. Fourth, security must be part of the invocation workflow rather than an afterthought.

These patterns also expose trade-offs. Function-centric names are expressive but can be difficult to deploy. Serverless placement is flexible but requires orchestration. RPC-style invocation is easy to understand but can favor one-to-one execution. Broker-based systems decouple time but may introduce infrastructure dependencies. NDNSF’s design point is to combine an RPC-like developer API with NDN’s data-centric names, signed Data, Sync-based publication, and application-visible provider selection.

This chapter supports RQ1 by turning the related-work review into a set of framework requirements rather than a survey checklist. It also prepares RQ2 and RQ3 by identifying which properties must be provided by the NDNSF runtime instead of being reimplemented separately by each UAV or inference application.]

The comparison will test this connection under explicit assumptions, including cached retrieval and replacement execution. It will not infer that a prior system lacks a mechanism merely because the mechanism is not summarized in a short description; if an existing system performs the same checks, the claim will be narrowed to the demonstrated integration or trade-off.

[Earlier material in this topic; Origin PDF p. 23, 24: The remaining gap is that existing NDN service systems have not provided a general-purpose runtime that combines:

- service containers that can be evaluated in realistic settings.

Table 2 summarizes the open issues motivating NDNSF.]

The novelty test is whether an existing mechanism already performs these checks on assigned producers, intermediate objects, and replacement executions under comparable assumptions. General workflow scheduling or waiting for dependencies is not sufficient evidence of a new protocol.

[Earlier material in this topic; Origin PDF p. 25: NDNSF is designed to address these issues using NDN names, verifiable Data, sync-based publication, and service-level authorization.]

Table 3: Related precedents and the question to be tested for NDNF. This is an evidence map, not a performance comparison.

System	Published mechanism / scope	Question to be tested and limit
NFN/NFaaS [6, 7]	Named functions, cached computation, and placement	Check whether their execution model supplies the task-specific producer and dependency admission required here
NSC/DNMP [9, 10]	Signed invocation, Trust Schema validation, and role-constrained command authorization	Compare standing caller/role permission with a role assigned after runtime discovery
CFN/CaSCON [21, 22]	Computation graphs, worker placement, and cache-assisted composition	Compare explicit object acceptance and replacement semantics under the selected assignment

[Earlier material in this topic; Origin PDF p. 25: Table 2: Open issues in existing service-oriented frameworks. Issue Why It Matters Endpoint binding Mobile services and intermittently connected providers should be invoked by name and capability rather than by stable host address. One-to-many invocation Reliability and collaboration often require multiple providers to hear a request and allow the user to select one or more of them. Data-centric security Data should remain signed, verifiable, and optionally encrypted at rest, not only while moving through a secure channel. Provider authorization A user must verify not only that it is allowed to call a service, but also that the selected provider is authorized to offer it. Provider selection Provider metadata such as load, queue size, location, and availability should be part of the service workflow. Large data Models, video, images, logs, and intermediate tensors should not be forced into small request payloads. Collaboration Tasks may require multiple providers, compensation requests, or aggregation of partial results. Operational validation Real systems need repeatable certificate, routing, configuration, and experiment setup to validate the framework outside toy examples.]

2.4 Comparing Authorization Designs under the Same Requirements

2.4.1 DNMP as a Concrete Trust-Schema Design

DNMP uses role-constrained signing to authorize measurement commands [10]. Its Network Key is the trust anchor for the role-key, device-key, and distributed-schema signing hierarchy. Given target, probeType, probeArgs, and the user’s roleType, the client finds a schema-permitted signing key among the user’s available keys. It rejects a missing match and selects the least privileged key if several match. The receiving Network Observer Daemon (NOD) dispatches a probe only after publication validation under the schema. Thus, this is command authorization, not merely verification of an arbitrary valid signature. DNMP also bounds publication lifetimes and retains

recent publication state for replay protection, assuming loosely synchronized clocks.

2.4.2 Adapting the Alternative to NDNF

For comparison, we propose retaining the NDNF Request–ACK–Selection–Response workflow while replacing ABE-backed invocation checks with role-constrained signing rules. The alternative must authorize both User invocation and Provider participation, match messages to active requests, and permit execution only after Selection. These adaptations are not features attributed to DNMP itself. Both designs require descriptor confidentiality before selection, recipient-scoped input/result protection afterward, and checks that the required service permissions have not been withdrawn.

[Old; Origin PDF p. 23: The open issue is integration with the service workflow. If access control is bolted onto individual applications, every application must separately handle permission retrieval, encryption attributes, message verification, and replay protection. NDNF makes these part of the runtime: ServiceController distributes permissions, service messages are protected according to service-level attributes, and one-time user/provider tokens bind ACK, selection, targeted fast-path, and response messages to the current invocation.]

[Earlier material in this topic; Origin PDF p. 34: • replay protection: whether one-time tokens have already been consumed.]

Table 4: Alternatives under the same discovery and execution requirements.

Design	Execution decision	Non-public discovery	Management cost
DNMP-inspired alternative	Validate role-constrained signatures; check invocation state and selected Provider	Combine authorization with service-group encryption (shared-key encryption for a service-authorized group) or ABE for discovery	Role credentials, trust rules, status or expiry checks, and encryption keys
NDNF ABE-backed	Check signatures, RequestNonce, ProviderChallenge, current permissions, and request state	Encrypt a descriptor for a service-authorized receiver set without listing current candidates	Identity credentials, ABE policies/DKEYs, status distribution, and rekeying

[Earlier material in this topic; Origin PDF p. 33: Figure 4: Example provider and user access-control policies in NDNF.]

Role granularity is policy-defined: a role can cover multiple actions, and credentials can be reused where the rules permit. DNMP is therefore a concrete authorization precedent, not evidence that every service needs a new signing certificate.

This comparison concerns complete designs, not measured DNMP-versus-NDNSF performance. The invocation-state and encryption choices must be implemented and evaluated under matched requirements before claiming a cost advantage.

The confidentiality column requires a separate protection mechanism: Trust-Schema signing rules do not encrypt discovery descriptors. Recipient encryption protects selected exchanges; service-group encryption or ABE can protect discovery before participants are selected. Trust Schema alone does not provide confidentiality, but it can be combined with either mechanism.

The proposed alternative requires current revocation-status checks or certificate expiry. Deleting a certificate from a repository does not invalidate cached copies. We do not attribute NDNSF's runtime revocation protocol to DNMP.

2.5 Protected Discovery and Permission Management

NDNSF uses NAC-ABE to protect Data associated with named service permissions. In the evaluated design, NAC-ABE uses a key-policy ABE (KP-ABE) construction: ciphertexts carry service attributes, while each decryption key (DKEY) carries the holder's access policy. The enhanced NAC-ABE library supplies the KP-ABE operations required by this construction [23]. This library is an implementation dependency rather than a new NDNSF protocol contribution. With this construction, the User can publish an encrypted discovery descriptor for a service-authorized receiver class without enumerating currently reachable Provider identities, while the Controller issues policy keys defining decryption access. NDNSF connects these permissions to invocation by checking protected challenge values, message signatures under trust rules, and current authorization/request state. Content encryption supplies confidentiality; the combined checks govern participation and execution. Section 3.4.1 explains the reuse of permission management and its additional costs.

The comparison must include group-key or broadcast-encryption alternatives where appropriate. A stable, small group can make conventional group encryption attractive; known Providers and simple permissions can favor signed invocation with recipient encryption. ABE becomes attractive when recipients are identified by service entitlement before their current identities are known,

and multiple entitlements should remain independent of identity-signing credentials. Policy size, membership churn, encryption cost, and authority availability bound this argument.

Chapter 3

NDNSF Framework and Proposed Extensions

[Old; Origin PDF p. 26: NDNSF Framework]

This chapter describes the invocation and authorization design under evaluation, including refinements to input/result confidentiality and permission updates, followed by the collaboration extension. The initial prototype supplies the single-Provider invocation foundation; the extension adds role assignments and named dependency edges. Implementation status is identified where a proposed mechanism requires further integration or testing.

[Earlier material in this topic; Origin PDF p. 26, 27: 3.1 Design Goals]

NDNSF is designed around five goals.

5. Operational validation. The framework should support MiniNDN evaluation, selected real-machine experiments, certificate management, and reproducible configuration.

- user and provider permission distribution;
- NAC-ABE-based service-level access control;
- built-in provider-selection strategies;
- adaptive admission control;
- Python bindings for experiment and workload integration;
- MiniNDN evaluation against NSC and gRPC.

A paper describing this completed foundation has been submitted to ICNP. The proposal therefore treats NDNSF v0.1 as prior completed dissertation work and uses the remaining chapters to define how that foundation will be extended and validated through application-driven workloads.]

3.1 Components and Application Responsibilities

The framework separates application handlers from the machinery that discovers Providers, correlates messages, and checks invocation state. A `ServiceUser` manages outgoing requests and their callbacks. A `ServiceProvider` registers service handlers and processes selected invocations. A `ServiceController` manages permissions and authorization material. These are software roles: a ground

station can consume camera services while providing image analysis, and a compute node can request an upstream service while executing its own assigned work.

[Old; Origin PDF p. 27: Figure 3 summarizes the three runtime roles implemented in the baseline. The controller manages policies and distributes permissions, providers subscribe for authorized requests and publish service results, and users publish requests, collect ACK metadata, select providers, and verify responses. This role separation is the foundation for the later UAV and distributed-inference service-container work.]

[Earlier material in this topic; Origin PDF p. 26, 30: 2. Data-centric security. Service messages and permissions should be signed, verifiable, and encrypted where appropriate.

4. Collaboration and resilience. The framework should support multiple providers, compensation after partial failure, and cross-validation.

3.2 NDN SF v0.1 Baseline

- ServiceController, ServiceUser, and ServiceProvider runtimes;
- service request publication over NDN synchronization;

This API is intended to keep application data in application-defined request and response objects while the framework handles naming, publication, permission checks, tokens, provider selection, and response delivery.]

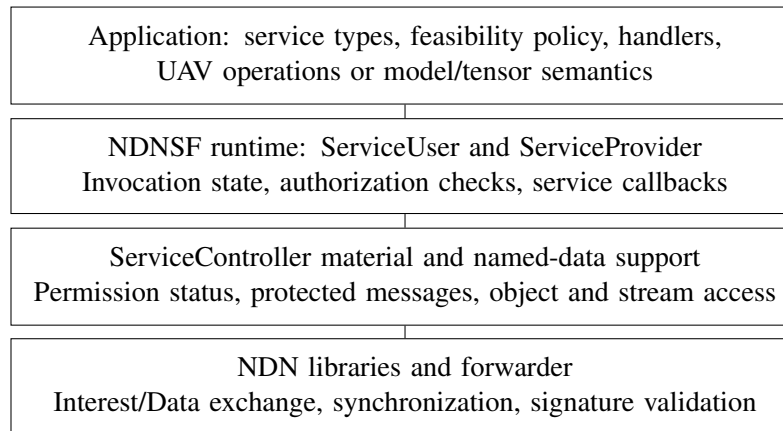


Figure 1: Responsibility groups in the framework. Lines denote software dependencies, not packet directions or a mandatory deployment topology. The Controller is a logical authority, not an on-path relay for every exchange.

[Old; Origin PDF p. 28: (a) Controller (b) Provider (c) User Figure 3: NDN SF v0.1 runtime roles used as the proposal baseline.]

Application code defines the meaning of a service and supplies its feasibility and execution logic. The framework cannot determine from a valid ACK whether a camera truly has an unobstructed view or whether an advertised GPU can meet a prediction deadline. It can associate the ACK with the request, verify the participating identity and required protected values, and apply the configured selection policy. A positive ACK expresses willingness subject to the request's condi-

tions; it is not evidence that execution has finished or a reservation unless the protocol explicitly provides one.

[Earlier material in this topic; Origin PDF p. 26, 27: 3. Dynamic provider selection. A request should be able to reach multiple providers and select the best one according to strategy.

- Request, ACK, Selection, and Response message flow;
- selective ACK and provider metadata support;

This baseline is substantial because it already connects the main pieces that a data-centric service framework must coordinate: service naming, service-message publication, user and provider authorization, encrypted service payloads, provider metadata, provider selection, and response verification. In other words, NDN v0.1 is not only an API sketch; it is an executable runtime that shows how NDN names, Sync, signed Data, NAC-ABE attributes, and service-level policy can be combined into one service-invocation workflow. This is why the proposal can focus on extending and validating the framework rather than starting from a blank design.]

The ServiceContainer supports composition of runtime roles and local services within a process. Its local registry gives trusted components a way to invoke registered application functionality without turning every internal call into a network exchange. The application must keep the runtime and callback dependencies alive until active operations are closed. Local composition is useful for shared camera or inference facilities, but remote callers cannot select it as an alternative authorization path.

[Earlier material in this topic; Origin PDF p. 26: 1. Application-facing simplicity. Developers should register and invoke typed services without writing service-specific NDN protocols.]

The current typed C++ interface illustrates this separation. The following shortened example uses application-defined, serializable status messages; identity provisioning, event-loop setup, and error handling are omitted. It illustrates application programming interface (API) use rather than a standalone executable.

[Old; Origin PDF p. 29: NDN v0.1 moved toward a generic dynamic API instead of generated, service-specific stubs. A provider can register a typed service handler:]

[Earlier material in this topic; Origin PDF p. 29: 3.4 Generic Dynamic API]

Listing 3.1: Typed registration and invocation of a compact status service.

```
provider.addHandler<StatusQuery, StatusReply>(  
    ndn::Name("/status"),  
    [](const ndn::Name& requester,  
        const StatusQuery& query, StatusReply& reply) {  
        readApplicationStatus(query, reply);  
    });  
  
user.RequestService<StatusQuery, StatusReply>(  
    candidates, ndn::Name("/status"), query,  
    [](const StatusReply& reply) { displayStatus(reply); },
```

[Old; Origin PDF p. 29: provider.addHandler<RequestT, ResponseT>(ndn::Name("/ObjectDetection/YOLOv8"), handler); A user can invoke a service through the generic runtime: user.RequestService<RequestT, ResponseT>(providers,]

[Earlier material in this topic; Origin PDF p. 30: ndn::Name("/ObjectDetection/YOLOv8"), request, onResponse, onTimeout, timeoutMs, strategy);]

```
    []() { reportTimeout(); }, 2000);
```

The handler interprets the query and fills the reply. The User interface supplies an invocation timeout and result callbacks, while NDN-SF manages the surrounding exchange. A service requiring confidential execution input uses the protected input-delivery path described below; this compact example does not prescribe placing arbitrary application input in discovery. Typed serialization removes some repeated application glue, but does not establish semantic compatibility between different service versions. That remains part of the service contract.

[Earlier material in this topic; Origin PDF p. 26, 27: NDN-SF v0.1 includes:

- a generic dynamic C++ service API;
- an NDN-SVS patch used by the evaluation path;]

3.2 Participants, Trust, and Exclusions

The preceding component description concerns software modules. This section uses User, Provider, and Controller as protocol and trust roles: the User requests a service and selects its Provider; in the

collaboration extension, it also coordinates role assignments. Providers offer and execute services.

A Controller manages service permissions, key distribution, and signed authorization status. These roles are not necessarily separate devices: one node may be both a User and a Provider.

[Old; Origin PDF p. 32: NDN uses a controller-authorized permission model. The ServiceController issues encrypted permission responses to target identities. Users and providers fetch their respective permissions, and permission Data is signed by the controller. NAC-ABE attributes map service messages to service-level authorization namespaces. One-time user and provider tokens protect the Request/ACK/Selection/Response flow against replay and confused-deputy behavior.]

[Earlier material in this topic; Origin PDF p. 32, 33, 34: 3.7 Security Model]

Figure 4 illustrates the intended policy model: provider permissions specify which named services an entity may offer, while user permissions specify which named services an entity may access. This proposal keeps that service-name-centered policy model in the UAV and distributed-inference extensions, while adding deployment procedures that make the trust root and publisher namespaces explicit.

- message integrity: whether a service message is signed and valid;]

The managed organization registers identities and approves service permissions; the Controller administers that policy. Participants require a provisioned trust anchor and validation rules for identity certificates and Controller-issued Data. The User may not know which enrolled Providers will respond to a particular invocation. Forwarders and caches can deliver Data without being trusted to authorize execution. The model trusts the Controller and the User's planning decision; it does not provide consensus against a malicious coordinator.

An attacker may observe visible names and traffic, inject invalid objects, replay previously valid messages, or attempt an operation without the needed service permission. Credential compromise and intentional sharing by an authorized principal require additional controls and are not prevented by ABE alone. Billing, arbitrary business transactions, globally optimal scheduling, and protection against all denial-of-service traffic are outside the claimed scope.

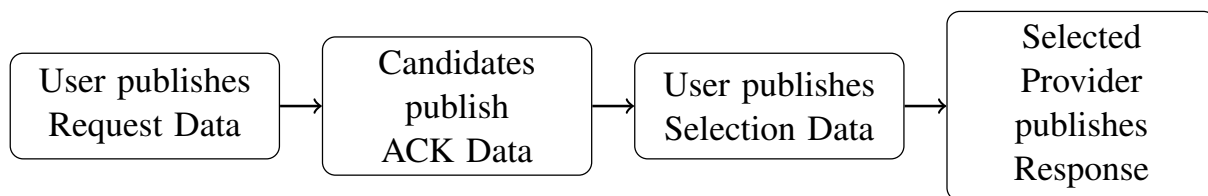
Authorization is permission to perform an operation, authentication establishes the signing identity under trust rules, and confidentiality limits who can decrypt protected content. Valid credentials do not prove honest load reports, correct inference, or truthful sensor observations. Signed status must be obtained and validated before it can affect local authorization; a disconnected participant cannot be assumed to learn revocation immediately. Deadline/expiry checks require an explicit clock policy. Tests must report the allowed skew and failure behavior rather than silently

assume synchronized clocks.

[Old; Origin PDF p. 23: NDN signs Data packets by design, but signatures alone do not answer all service questions. A service framework must decide who may request a service, who may provide a service, what data should be encrypted, how keys should be distributed, and how replayed messages should be rejected. Prior work on attribute-based encryption and NAC-ABE shows how fine-grained access control can be integrated with NDN data names [7, 21]. These ideas are central to NDNSF.]

[Earlier material in this topic; Origin PDF p. 23: 2.5.5 Security and Access-Control Mechanisms]

3.3 Four Named Message Types



Controller: authorization material and signed status. Sync: discover names. Interests: retrieve Data. User/Provider: validate Data, check permissions and request state before acceptance. Figure 2: Logical invocation dependencies. Arrows express application causality, not unsolicited network Data delivery.

[Old; Origin PDF p. 31: 7. The user verifies tokens, provider identity, and response data.]

[Earlier material in this topic; Origin PDF p. 31: 3.6 Service Invocation Workflow]

RequestNonce is a fresh User-generated application value carried in protected Request content. ProviderChallenge is a Provider-generated application value carried in protected ACK content and checked when the User constructs Selection. They are challenge values for this service exchange, not the NDN Interest Nonce used by the forwarding protocol.

[Earlier material in this topic; Origin PDF p. 31: 1. The user publishes a service request under a service name.]

Table 5: Logical message responsibilities; this is not a wire-format listing.

Data	Purpose	Principal checks
Request	Publish service requirements, minimized discovery descriptor, request identity, and RequestNonce	Validate signature and current authorization context; recover the protected descriptor before evaluating willingness
ACK	Report positive or negative participation decision; a positive ACK carries selection information and ProviderChallenge	User checks signer under trust rules, service permissions, request ID and returned RequestNonce; only eligible positive ACKs enter Provider selection
Selection	Notify the chosen Provider and supply protected input or input references and request-scoped keys	Provider validates User signature and Selection using stored ProviderChallenge; checks intended Provider, request state, and service permissions
Response	Publish completion/result metadata or references to protected results	Check authorized producer, request context, result identity, and decryption scope

[Old; Origin PDF p. 30: Table 3: NDNSF v0.1 service messages. Message Producer Purpose RequestMessage User Publishes a named service request and application payload. RequestAckMessage Provider Reports whether the provider is willing to serve the request and exposes runtime metadata for selection. SelectionMessage User Selects one or more providers and binds the request to the selected provider tokens. ResponseMessage Provider Returns the service result from a selected provider.]

[Earlier material in this topic; Origin PDF p. 30, 31: 3.5 Service Message Model

Table 4: Representative fields carried by the four message types. Message Representative Fields RequestMessage service name, request ID, requester identity, payload, UserToken RequestAckMessage service name, request ID, provider identity, status, message, metadata payload, echoed UserToken, ProviderToken SelectionMessage service name, request ID, selected provider identity, ProviderToken ResponseMessage service name, request ID, provider identity, result payload, echoed UserToken]

The protected challenge exchange is explained below. The table describes the single-Provider invocation. Plan identity, role assignments, and dependency checks are added by the collaboration extension below, not by the initial four-message flow.

[Earlier material in this topic; Origin PDF p. 31, 32: The message fields are intentionally service-independent. Application-specific data remains in the payload or in typed request/response objects, while the framework uses service names, request IDs, identities, and one-time tokens to provide authorization checks, replay resistance, and response verification.

The normal NDNSF invocation workflow is:

2. Providers with permission to serve the service receive the request.

6. Selected providers execute the request and publish responses.

This workflow supports both one-to-one and one-to-many service models. The same request can reach multiple providers, enabling the user to select the first available provider, randomly balance load, select all providers, or use a custom strategy based on metadata.]

These are NDNSF Data objects. Sync supports discovery of their names, after which Interests retrieve Data, possibly from a cache. A negative ACK is an application participation decision, not a network Nack. Not every observed Request must receive a reply: invalid, unauthorized, duplicate, or expired messages are subject to validation and state checks. A positive ACK expresses willingness; it does not itself start the selected task or guarantee later resource availability.

[Earlier material in this topic; Origin PDF p. 31: 3. Providers decide whether to ACK based on availability, local state, and application-defined metadata.]

3.4 Authorization and Confidentiality

3.4.1 Rationale for ABE-Backed Service Authorization

NDNSF uses ABE to protect discovery descriptors and reuses attribute-based service permissions in its invocation checks. The first choice addresses confidentiality before the participating Providers are known; the second keeps service entitlements independent of identity-signing credentials. Neither implies that ABE is always preferable to role-based authorization with suitable encryption. Withdrawal and rekeying are lifecycle costs to evaluate. Execution authorization determines whether a User may invoke a service; Provider authorization determines whether an entity may offer that service. Content confidentiality limits who can read protected content. The Controller administers these service permissions; the User and Provider check them during invocation. Signature validation under trust rules and invocation-state checks remain necessary. Permission to offer a service is distinct from selection for one particular task.

[Old; Origin PDF p. 34: • user authorization: whether the user may request a service;]

[Earlier material in this topic; Origin PDF p. 34: • provider authorization: whether a provider may serve a service;]

The design addresses three requirements: keep discovery descriptors non-public before participating Providers are identified; authenticate participants, verify their service permissions, and

reject replayed invocation messages; and support permission grants and withdrawals during operation. Permission aggregation is an additional management benefit, not a fourth security guarantee. Reason 1: confidential discovery before Provider selection. Before Provider selection, the User publishes a discovery descriptor containing the location, deadline, and resource requirements needed for a Provider's positive-ACK decision. For example, nearby UAVs need an observation area and arrival deadline to decide whether to participate, but do not yet need the full application input. The User encrypts this descriptor using ABE and the requested service's attribute, without enumerating Provider identities. Confidentiality is against parties lacking the required decryption capability: authorized but unselected Providers can still read the descriptor. Content encryption does not hide outer Data names or traffic patterns. In the request-scoped confidentiality path, the full application input is encrypted with a request-scoped key; Selection protects that key for the selected Provider. The requesting User retains access. A Trust Schema alone does not encrypt content; a role-authorized design can also use ABE or service-group encryption (encryption under a shared group key) to provide discovery confidentiality.

[Earlier material in this topic; Origin PDF p. 34: • data confidentiality: whether the payload is encrypted for authorized participants;]

Encrypted Request Data can be retrieved or cached without granting decryption access. This separation of delivery from disclosure also holds for other content-encryption schemes; it is not unique to ABE. Reason 2: reusing attribute-based permissions for service invocation. Confidential discovery alone does not justify ABE-backed execution authorization: ABE encryption could instead be combined with role-based invocation checks. NDN-SF chooses to administer both Provider and User service permissions through the existing attribute-authority infrastructure, separately from identity-signing credentials.

Provider permissions determine access to RequestNonce; User permissions determine access to ProviderChallenge. NDN-SF checks RequestNonce in ACK Data and validates Selection Data using the stored ProviderChallenge. Signature, current-permission, and request-state checks remain separate requirements (Section 3.4.2). Decryption is therefore one part of the invocation check, not permission to execute any request.

DNMP instead uses role signing keys and a Trust Schema to constrain command targets and actions [10]. In the DNMP-inspired alternative (Section 2.4.2), retaining ABE for discovery would require managing both decryption permissions and role-based invocation permissions. NDNFSF uses the attribute-authority infrastructure for both service-level checks, without adding role signing credentials solely to represent service membership. Trust rules for validating Data signatures remain necessary.

The evaluation will measure the costs and benefits of this shared management infrastructure. Provider and User permissions remain distinct; User permission decryption keys and challenge processing add costs beyond descriptor encryption. For policies requiring command-argument restrictions, DNMP's schema-based checks may be a better fit than NDNFSF's service-level checks. The evaluation must compare both designs at the same policy granularity, including credential updates and invocation overhead. Reason 3: service-permission aggregation as an additional management benefit. KP-ABE associates attributes with ciphertext and an access policy with each decryption key (DKEY) [23, 24]. NDNFSF uses /SERVICE/<service> for Provider access and /PERMISSION/<service> for User access. Within one authority and parameter generation, an entity's service permissions can be combined in one individually issued DKEY. For example, one User's policy can allow detection OR mapping, while another allows detection only. Their identity-signing credentials need not change to represent these different service sets; each service retains its own ciphertext attribute.

Compared with DNMP's role-key-based authorization [10], NDNFSF represents service entitlements in the DKEY rather than in additional role-specific signing credentials. Where equivalent entitlements require several distinct role credentials, this avoids issuing and maintaining those additional signing credentials. The comparison depends on role granularity: one DNMP role can permit several commands, and Trust Schemas can support other credential arrangements. The evaluation will therefore measure independently managed credentials alongside DKEY size, policy complexity, distribution, rekeying, and runtime costs.

3.4.2 Authorization Mechanisms, Lifecycle, and Costs

Identity authentication. The User and Provider validate Data signatures and certificate chains under the applicable trust rules. This identifies an accepted signer; it does not alone establish that signer's permission to offer or invoke the requested service. Service-permission verification. In the ABE-backed invocation path, the User generates a fresh random value, RequestNonce, and includes it in ABE-protected Request content. This is an NDN SF application-level value, not the NDN Interest Nonce used for forwarding duplicate and loop handling. A Provider that recovers it returns it in ACK Data; the User compares the returned value with the one stored for the active request. Thus RequestNonce supports both request binding and a check of access to the service-protected value. The Provider generates ProviderChallenge and includes it in ABE-protected ACK content. The User uses the recovered ProviderChallenge to construct Selection Data. Before execution, the Provider validates Selection Data using ProviderChallenge stored for that request. Together with current service-permission checks, these checks determine whether the Provider may offer the service and whether the User may invoke it.

[Old; Origin PDF p. 32, 33: NAC-ABE enforcement is tied to the invocation tokens rather than to a separate permission list carried in every service message. A Request message includes a fresh UserToken encrypted under the service attribute, so only an authorized provider can decrypt the request and return that token. The ACK echoes the UserToken and carries a fresh ProviderToken encrypted under the permission attribute. The user proves selection authority by returning the selected provider's ProviderToken in the Selection message, and the final Response echoes the UserToken. In this way, NAC-ABE attributes gate who can decrypt each authorization token, while one-time tokens bind that authorization check to a single service invocation.]

These checks demonstrate access to protected challenge values, not the creation of new permissions or a formal proof of exclusive key possession. Authorized key or challenge sharing is outside the claimed threat protection. This reuses attribute-based permission management rather than a separate set of role-based invocation rules. It is not cost-free: confidential discovery already needs Provider DKEYs, whereas User permission DKEYs and challenge processing are additional authorization costs. ABE confidentiality combined with DNMP-inspired role authorization remains a valid alternative. Replay protection. Participants match messages to an active request and its protocol state. The Provider records whether ProviderChallenge has already been used to accept

Selection Data for execution; receiving the same Selection again must not start another execution. This requires appropriate retention and invalidation of request state, including across restart. Neither encryption nor possession of a challenge value alone prevents replay. These admission checks do not establish exactly-once application side effects across crashes. Runtime permission grants and withdrawals. PolicyStatusData denotes a Controller-signed named Data payload, not a grant by itself. It identifies the service and current ControllerVersion, validity interval, policy digest, optional ABE public-parameter name and digest, the Controller certificate, and revocation targets. Participants validate the enclosing Data signature and this status before updating local permission or DKEY state; the status distributes and authenticates policy state but does not independently grant permission.

The Controller updates permission status for both operations. An ordinary grant updates the target entity's complete permission policy; that entity retrieves a replacement DKEY without changing the ABE public parameters. Withdrawal additionally rotates the Controller's ABE master key and public parameters. Entities retaining permissions obtain replacement DKEYs encoding their remaining permissions. The master key stays with the Controller. Rotation is Controller-wide, not confined to the withdrawn service.

In the proposed update-discovery path, which is not yet fully verified end to end, the Controller publishes updated public parameters as signed named Data and uses Sync to announce their availability. Participants retrieve and validate the parameters and Controller-signed PolicyStatusData, then refresh their DKEYs as required. Ordinary grants also require participants to discover and retrieve updated permission status, even though the public parameters have not changed. Sync is an update-discovery mechanism, not the authority for granting permission. The local implementation supports status retrieval and parameter rotation; the dedicated Sync announcement path remains to be verified end to end.

Current-permission checks enforce withdrawal after the updated status has been retrieved and validated. Parameter rotation restricts exchanges encrypted under the new generation. Disconnected nodes cannot apply updates immediately, and previously disclosed keys or plaintext cannot

be taken back. Lifecycle comparison and evaluation. DNMP specifies validity bounds and replay-state handling [10]; these do not by themselves provide immediate withdrawal of an unexpired role key. For the comparison, a DNMP-inspired design can revoke the relevant credential and require execution nodes to check signed revocation status or reject expired certificates. The execution node then rejects the revoked or expired credential; distributed certificate copies need not be retrieved. Both approaches require update propagation, and neither retracts previously disclosed keys or plaintext.

Focused integration tests exercise NDNSF's runtime revocation checks, including identity-, certificate-, and service-scoped withdrawal. These tests establish behavior at the exercised runtime boundaries, not network-wide instantaneous revocation or lower cost than certificate-based alternatives. The remaining evaluation should measure status-propagation delay, rejection delay, rekeying traffic, and effects on unaffected members. Runtime status validation and key rotation support runtime permission changes, with explicit propagation and rekeying costs. They are not ABE-exclusive capabilities. NDNSF introduces ABE for confidential discovery and reuses its service-permission management for invocation checks and permission updates; signatures and request-state checks remain necessary, separate parts of the design.

Discovery confidentiality applies to descriptor Content against parties lacking the required ABE decryption capability. It does not conceal the descriptor from other authorized candidates or provide traffic-analysis resistance. In the request-scoped protection design, the User encrypts application input with a symmetric key and delivers the necessary key material through Selection, encrypted for the chosen Provider's authenticated public key. The response key is shared by the requesting User and selected Provider. Other Users cannot decrypt the protected result under the key-secrecy assumptions; the Provider necessarily knows the plaintext it produces. These are the protection requirements for the evaluated design, rather than claims about every historical implementation path.

[Earlier material in this topic; Origin PDF p. 33: The current model separates:]

3.5 Candidate Information, Selection, and Admission

Selection is a policy decision over eligible candidates, not a substitute for permission checks. The current User implementation exposes first-positive-ACK, random-positive-ACK, all-positive-ACK, and application-supplied selection policies. Selecting several Providers for independent executions does not, by itself, assign complementary roles or enforce dependencies. The proposed collaboration extension adds those semantics; they should not be attributed to the initial single-Provider invocation workflow.

[Old; Origin PDF p. 32: The implemented provider-selection strategies are deliberately generic. The First Responding strategy selects the first successful ACK, which is useful when low latency is the main goal. Random Selection chooses among valid ACK candidates, which can spread load without requiring detailed provider state. All Selected chooses every provider that returned a valid ACK, which supports cooperative or redundant execution. A Custom Selection strategy runs after the ACK collection window and can use ACK metadata such as latency, queue length, load, location, model availability, safety state, memory, or accelerator capacity. The key design point is that readiness reporting and final provider choice are separate framework steps: providers decide whether and how to ACK, and the user-side strategy decides which ACK candidates should be selected.]

[Earlier material in this topic; Origin PDF p. 31: 5. A selection strategy chooses one or more providers.]

Early ACK arrival is an operational selection rule, not proof of the lowest execution latency.

Random choice can serve as a controlled policy baseline; an application policy can

[Earlier material in this topic; Origin PDF p. 31: 4. The user collects ACKs during an ACK window.]

Table 6: Information needed by selection and its limits. Application-specific fields are service contracts, not mandatory fields in every ACK.

Information	Decision it supports	Interpretation limit
Request/service identity and deadline	Associate a candidate with the requested operation	A matching identifier does not establish permission
Positive/negative ACK and reason	Distinguish willingness from explicit refusal	No ACK may instead reflect loss or delay
Service-specific capability and resource report	Filter unsupported models, sensors, memory or timing constraints	Signed reporting proves provenance, not the truth of a capacity prediction
Available artifacts or compatible state	Estimate preparation cost before selecting a Provider	Reuse still needs version and authorization checks
Provider identity and protected challenge information	Validate the participant and selected exchange	Neither guarantees future availability

consider estimated completion time or preparation cost. Comparisons should hold the collection window and eligibility checks fixed when isolating the policy’s effect. Collaboration must account for dependencies and compatible assignments, not merely choose the individually fastest candidate for every role.

Selection identifies the intended executor. Execution-time admission determines whether that Provider can still accept the work under its current resource limits. The lifecycle and failure outcomes below distinguish refusal, queued work, execution, and completion.

[Earlier material in this topic; Origin PDF p. 30: NDNSF v0.1 uses four framework messages to separate service publication, provider admission, provider selection, and result delivery. This message model is central to the one-to-many service design: a request can be observed by multiple authorized providers, providers can expose availability and metadata before execution, and the user can explicitly bind the request to selected providers.]

3.6 Invocation Modes and Execution Lifecycle

3.6.1 Discovery, Known Providers, and Local Composition

Normal discovery is appropriate when the executor has not been chosen. The User chooses among positive ACKs that pass validation and policy checks. Selection timing follows the configured policy rather than one universal waiting rule.

[Earlier material in this topic; Origin PDF p. 36, 37: 3.9 Limitations of NDNSF v0.1

- Operational validation: v0.1 demonstrates the security model, but broader validation needs clearer certificate bootstrapping, trust-schema guidance, configuration procedures, and reproducible experiments.

3.10 Supporting Mechanisms for Provider Collaboration

The following mechanisms support provider collaboration in specific operating conditions. They are motivated by the UAV and distributed-inference workloads, but they are not separate dissertation threads and should remain generic framework mechanisms rather than application-specific shortcuts.]

A supported first-positive-ACK path may select before timeout. A collection-based policy waits until its declared closure condition is met. The proposed snapshot-based collaboration commits only after its candidate collection closes.

A negative ACK communicates non-participation; silence cannot distinguish refusal from loss or disconnection. Selecting several independent executions does not assign complementary roles and dependencies as a collaboration plan does.

[Earlier material in this topic; Origin PDF p. 37: • Provider collaboration: v0.1 supports selecting providers, but larger tasks need collaboration among multiple providers, partial failure handling, and compensation requests.]

Known-Provider invocation is appropriate when the operation must use a particular device, for example, stopping the camera that was previously started. The current `RequestServiceTargeted` path can reuse previously bootstrapped one-time authorization material to avoid repeated ACK/Selection exchanges under its supported conditions. It still checks current authorization and request state. The fast path is not universal: request-scoped confidentiality needs a Selection envelope to carry the input and response keys, and the current implementation uses the bootstrap/ACK/Selection path for that case. The explicitly Selection-gated input mode is rejected by the Selection-free Targeted interface. Thus, targeting a known Provider is an application requirement, whereas omitting exchanges is a conditional implementation optimization.

[Old; Origin PDF p. 37, 38: Some services, such as UAV flight-control commands, are directed at a known provider and must minimize latency. The proposed Targeted invocation path keeps NDNSF permissions, one-time user/provider tokens, and replay protection, but avoids repeated ACK/Selection rounds after a token batch has been bootstrapped through the normal flow. This gives known-provider commands a fast path while keeping the security semantics of normal NDNSF invocation.]

[Earlier material in this topic; Origin PDF p. 36, 37: • Low-latency known-provider control: the

normal Request/ACK/Selection/Response workflow is appropriate for provider discovery, but control commands to a known drone or device need a faster Targeted path while preserving authorization and replay protection.

3.10.1 Targeted Invocation]

Trusted local composition operates within a process through the local registry. It has neither remote discovery nor the same network threat boundary. Evaluation should identify whether a handler was reached locally, by normal discovery, or by a targeted exchange. Otherwise an apparent invocation improvement might merely measure a different operation.

[Old; Origin PDF p. 38: A service-container application may contain several services in the same trusted process. A local call from one internal service component to another should not need to publish an NDN request. The proposed local invocation path therefore provides synchronous and asynchronous in-process invocation for explicitly registered local services. It is not a wire-protocol mode: remote callers cannot request it, and it bypasses NDNSF network security only inside the trusted local container boundary.]

[Earlier material in this topic; Origin PDF p. 37, 38: • Service-container composition: real applications often contain multiple services in one trusted process. Those services need efficient local composition without exposing a network bypass to remote callers.

3.10.2 Trusted Local Invocation]

3.6.2 Admission, Progress, and Failure

An API submission, admission to a runtime queue, acceptance by a Provider, and completion of application work are different events. Under overload, reporting latency only for completed requests can hide dropped or rejected work. The runtime therefore needs observable outcomes for refusal, timeout, and terminal results, while an evaluation must count all submitted requests in its stated denominator. Selection-status queries can help determine the observed status of selected work; they are not proof of a physical effect or a distributed transaction.

Applications must also distinguish cancellation of interest in a result from reversal of an action. A recording can be stopped, but previously recorded frames still exist. An inference worker can stop producing new output, but a completed remote action cannot be undone by deleting requester state. Retrying requires the application's stated idempotency or compensation policy. For the proposed collaboration extension, replacement additionally requires old assignments and late callbacks to stop advancing the current plan.

3.7 Control Messages, Streams, and Complete Objects

Small service messages communicate intent, permission evidence, and status. Large application data should not be copied into each control message. An object reference allows a service to identify a model fragment, recording, or image, while the recipient retrieves its named Data separately. The reference and service contract must supply enough information to select the intended version and validate the reconstructed object. The concrete interface determines the available name, size, digest, segmentation, and protection metadata; not every object type needs an identical collection of fields. [Earlier material in this topic; Origin PDF p. 37, 39: • Large artifacts: v0.1 service payloads are not intended to carry models, recordings, images, or intermediate tensors. Applications need typed references to named Data objects plus native NDN segmented retrieval for large signed and encrypted objects.]

This design can optionally use persistent storage, but storage is an implementation support component rather than a separate research direction in this proposal. If persistent storage is used, it stores opaque NDN Data segments that have already been named, signed, and optionally encrypted by the application or by a framework helper. Security is not handled by a storage-specific path: NDN's normal hybrid encryption, signatures, tokens, permissions, and trust-schema validation protect the service message and the referenced Data. Discovery metadata can still accelerate validation, but large-object transfer itself is based on typed object references and native NDN segmented retrieval.

This chapter supports RQ2 by previewing the reusable NDN mechanisms that the dissertation will refine: secure invocation, provider authorization, provider collaboration, and its supporting mechanisms. It supports RQ3 by identifying where collaboration introduces reliability, security, and latency tradeoffs, and where supporting mechanisms are needed to control overhead without weakening the core service semantics.]

The important separation is between identifying an object, obtaining its bytes, and authorizing its use. A digest can detect a mismatch with the expected bytes only when the expected digest itself is trusted. A signature binds Data to a key under the application's trust rules. The invocation or collaboration checks establish whether that producer's object is permitted as an input to the current execution. Recipient-scoped key delivery controls access to protected Content. None of these checks should silently substitute for another.

[Old; Origin PDF p. 38: Models, video, images, activations, and logs should not be placed in small request payloads. The proposed pattern is to let the producer segment the object as named NDN Data and place only a typed object reference in the NDN request or response. The reference can include the object name, object type, version or session id, expected digest, expected size, final segment information, source type, and optional encryption metadata. The receiver then uses normal

NDN segmented fetching, such as ndn-cxx segment fetch logic, to retrieve the object by name.

This abstraction is intended to be a generic NDNSF mechanism, not an API for a particular storage service. Small request and response payloads remain useful for control messages and compact typed objects. When the payload becomes a large artifact, the same logical service invocation should carry a reference rather than the artifact bytes. The application or helper can turn a payload into named Data segments, sign and optionally encrypt those segments, and then pass a typed reference through NDNSF. This keeps service selection, authorization, token checks, and timeout diagnosis in the normal NDNSF path, while large bytes move through NDN's native named-data retrieval path.]

[Earlier material in this topic; Origin PDF p. 38: 3.10.3 Large-Data Reference Abstraction]

An ongoing stream has a different lifecycle from a complete object. Live video may prefer recent frames and tolerate explicitly identified gaps. A model artifact generally requires every needed byte before loading. Incremental inference requires ordered application events and a terminal outcome, but its token stream is not necessarily interchangeable with a camera stream interface. NDNSF supplies named-data mechanisms; applications define acceptable gaps, buffering limits, and completeness rules. The evaluation must measure the chosen policy, including its discarded data and queueing costs.

Named-object reuse is valuable in the multi-view case: the compute Provider returns annotations and references to the original images, and provides the requester with the image content keys it is permitted to receive. The requester can retrieve those images from available producers or caches and validate their original signatures. This avoids producing a second image object merely to change the receiver. It does not guarantee that a cached copy remains available; retention requirements need an explicit producer or repository policy.

[Old; Origin PDF p. 39: The same reference format should be usable across UAV recordings, camera artifacts, model shards, runtime artifacts, intermediate activations, and logs. As a result, UAV-APP and DistributedInference can share one framework-level large-data vocabulary: object type, object name, producer, session or version, size, digest, final segment hint, and optional encryption metadata. This avoids embedding application-specific conventions such as raw Data-name strings inside service payloads and makes large-object behavior inspectable by applications that do not understand NDN packet details.]

[Earlier material in this topic; Origin PDF p. 36, 37: NDNSF v0.1 establishes feasibility, but it does not yet fully address the requirements of dynamic service applications beyond controlled framework tests. The main limitations that motivate the remaining dissertation work are:

- Application-level validation: v0.1 evaluation shows the core framework works in MiniNDN, but the dissertation must evaluate whether the design supports demanding UAV and

distributed-inference workflows.]

3.8 Proposed Extension: From ACKs to a Collaboration Plan

For this proposal, a candidate snapshot is the canonical, ordered set of validated eligible Provider offers observed when the ACK collection window closes. Each record includes the Provider identity and the signed offer/ACK information used for eligibility, such as its service capability and validity data. The snapshot digest is a cryptographic digest of that canonical encoding. A logical request is identified by its request ID; an execution attempt ID distinguishes one run of that request, including a replacement. The execution identity is the request/attempt pair. The snapshot is immutable for that attempt; a replacement receives a new attempt and must not silently reuse the old assignment.

The current NDNFSF implementation already closes candidate collection, checks the assigned snapshot digest, verifies that selected Providers belong to that snapshot, and rejects conflicting plan commits. These mechanisms provide an implementation basis for the collaboration study. The remaining research evaluates dependency acceptance, recovery, and cross-application behavior against the acceptance rules defined below.

The extension changes the selection problem from choosing one executor to assigning complementary task roles. The application supplies a role/dependency template, required services, feasibility constraints, and deadline. The template may be written manually; automatic generation is not necessary to establish the framework contribution. A replaceable planning policy selects among eligible positive ACKs after the collection window closes. Closing the window freezes the candidates observed for that decision, not all possible Providers in the network. ACK claims can become stale, so a selected Provider still performs execution-time admission and may reject work it can no longer execute.

[Earlier material in this topic; Origin PDF p. 29: This collaboration model is intentionally application-independent. Multi-UAV patrol can use it to assign mission segments and recover from partial failure, while distributed inference can use it to assign model roles and coordinate intermediate artifacts. In both cases, NDNFSF should provide the generic service semantics: request publication, provider metadata, provider selection, authorization, status diagnosis, and response verification. The application defines the domain-specific work; the framework defines how selected providers participate in one logical service workflow.]

In the proposed design, the User signs a Plan Manifest containing the request identity, candidate-snapshot digest, exact role-to-Provider assignments, permitted named dependency edges, authorization/key scope, and execution identity. Its plan digest is computed over a canonical plan body that excludes the digest field itself. Providers validate that signed plan before deriving local dependency checks; a long-term certificate does not specify this task's assignments. The User must record the committed plan and reject a different plan for the same execution identity. This is a local state-machine rule under the trusted-User model, not a consequence of signing alone or protection against a Byzantine User signing two plans.

[Old; Origin PDF p. 29: The central extension beyond NDNSF v0.1 is provider collaboration. In a collaborative service, one logical request may involve multiple authorized providers rather than a single endpoint. Providers may advertise ACK metadata such as availability, queue state, location, or role capability; the user may select one or more providers after an ACK collection interval; and the application may issue compensation requests when only part of a logical task is completed. Long-running selected work should also be diagnosable: after a timeout, the user should be able to learn whether a selected provider has not seen the selection, is queued, is executing, has failed, or has already published a response.]

[Earlier material in this topic; Origin PDF p. 29: 3.3 Provider Collaboration as the Central Extension]

The technical difficulty is that the abstract dependency graph can be known before discovery, but its concrete signing identities and object namespaces cannot yet be fixed. The proposed protocol therefore separates an application-supplied template from its post-ACK binding. Each consumer derives its permitted predecessor, object-name constraints, and key scope from the same committed assignment. A later ACK cannot change only one consumer's interpretation. This is stronger than waiting until an arbitrary input is available, but does not establish optimal placement or agreement among malicious planners.

3.9 Proposed Dependency Admission and Execution State

Under the proposed acceptance rules, a downstream Provider checks that the execution is active and within its deadline before accepting a dependency. The Data signature must validate under the applicable trust rules. Newly produced inputs must match the assigned request, plan, execution identity, predecessor, and declared edge; historical inputs instead require the explicit reuse autho-

rization described below. Name, format, and key-scope constraints still apply to both. Required segments must validate, and incomplete or malformed objects cannot satisfy an edge. The same checks apply whether Data is retrieved from a producer, a cache, or a repository. The plan references objects in their actual producers’ authorized namespaces rather than republishing them under a different producer identity.

Table 7: Proposed task-state contract and required negative checks.

State/event	Permitted transition	Required rejection or bounded behavior
Collecting ACKs	Close window and validate candidate set	Late ACK does not silently alter a committed plan
Selected	Validate Selection and perform local admission	Positive ACK alone, wrong Provider, or already-used ProviderChallenge cannot start work
Waiting for dependencies	Execute when required authorized edges are satisfied	Wrong role, undeclared edge, incomplete object, or wrong active execution is rejected
Completed/cancelled	Return recorded status under duplicate-message policy	Duplicate publication does not implicitly repeat a side effect
Failure/reassignment	Explicitly terminate or authorize a replacement execution	Old execution must not acquire authority from the new assignment; allowed result reuse is explicit

The proposed recovery rules separate retrieving an object again from repeating computation. Once a Provider applies a replacement, old execution messages and callbacks cannot advance the new execution.

An existing artifact is reusable only when the new signed plan explicitly references its exact immutable identity, original producer and provenance, destination dependency, and permitted key scope. The artifact retains its original name and signature; it is not relabeled as newly produced. The consumer checks that explicit reuse authorization instead of requiring the historical artifact to claim the new execution identity. All other old objects remain ineligible.

Application checks must additionally establish that a captured image is still timely or that a tensor/state object is compatible. Thus, an old artifact may remain useful without allowing messages from its old execution to advance the replacement. Application-defined compensation does

not provide distributed atomic transactions or exactly-once physical action.

RQ1 evaluates the authorization design and its alternatives. RQ2 tests the plan and state rules through wrong-role and wrong-edge inputs, replay, cancellation, and equivalent validation of Data obtained from producers and caches. These collaboration experiments extend the invocation evidence presented in Chapter 6.

3.10 Conditional Correctness Argument

The preceding admission argument is conditional on uncompromised signing keys, correctly enforced trust and authorization checks, and Providers that obey the specified state machine. To permit execution only after Selection, every path that starts a handler must check an active assignment and confirm that the one-time authorization state has not already been used to start execution.

To enforce the declared dependencies, every input path, including retrieval from a cache, must check the dependency edge and the assigned producer. Historical artifacts instead require explicit reuse permission. The artifact reference must identify exact Data, for example by a full name including its implicit digest or a validated segment manifest with content digests.

After replacement, old workers and late callbacks must not advance the new execution. Restart recovery must preserve or safely invalidate the relevant one-time state. A valid signature and cache FreshnessPeriod alone cannot establish these properties.

The study will test these conditional invariants by injecting a wrong-role object within the same request, an undeclared input from a correctly signed producer, and a late callback after replacement. Each case isolates a property that a request ID alone cannot express. The argument concerns protocol acceptance and execution state under the stated assumptions; application-result correctness and physical side effects require separate validation.

Chapter 4

UAV Application over NDN

[Old; Origin PDF p. 40: UAV Network Application over NDN]

NDN-UAV exercises service discovery, authorization, data delivery, and execution management through control, telemetry, video, recordings, and collaborative sensing. Single-Provider operations examine the invocation foundation; multi-view collection additionally tests complementary roles. Chapter 6 specifies the comparisons and measurements.

[Earlier material in this topic; Origin PDF p. 40, 41: • Command authorization. Flight-control commands must be restricted to authorized users and protected against replay.

• Data verification. Telemetry, images, videos, and mission results should be verifiable independently of the channel.

Table 7: Service types in the proposed UAV application. Service Type Purpose Flight control Arm, takeoff, land, goto, manual control, and generic MAVLink execution. Telemetry and state Report GPS, altitude, battery, armed state, flight mode, readiness, camera availability, link state, and mission state. Video control Start/stop live video, negotiate bitrate, and report actual streaming state. Video data Publish encoded frames or chunks as NDN Data for low-latency playback. Recording discovery Authorized discovery of locally recorded encrypted video objects. Recording retrieval Retrieve encrypted signed NDN Data objects through large-data references. Object detection Detect objects such as cars and trucks from camera frames, either at the ground station or onboard. Mission/patrol Assign waypoints, partition tasks among drones, start/stop missions, and compensate for missing subtasks. Discovery/status Publish installed service information and readiness state.]

4.1 Application Structure and Service Workflows

The UAV application connects an operator's ground station with drone-side service containers. Drone services expose camera operations, telemetry, and mission-related operations through NDN. The ground station can also provide an object-detection service, so the same application includes both service consumption and service provision. Application code interprets camera, vehicle, and mission data; NDN supplies the reusable invocation and data-delivery mechanisms.

[Old; Origin PDF p. 41, 42: The proposed NDNSF-UAV-APP models a UAV network as a service-container application. The ground station and each drone run containers with different service instances, and the APP-level configuration determines identities, service names, runtime roles, camera behavior, and flight-controller backend. Table 7 summarizes the service types.

The UAV application contains two main roles:

- Ground station role: operator interaction, map-based mission planning, telemetry and video observation, object-detection service composition, and NDNSF user behavior.
- Drone role: flight-control adaptation, telemetry publication, camera and video publication, encrypted recording support, mission execution, and NDNSF provider behavior.

The application is explicitly service-oriented. A drone does not expose one monolithic endpoint. Instead, it exposes independently named services for flight control, telemetry, video control, camera frames, recording discovery, and mission assignment. The ground station uses structured runtime state to represent telemetry freshness, mission progress, video availability, and safety readiness, so that operator-visible status and command decisions remain consistent across independently named services. Known-drone commands use Targeted invocation, while collaborative workflows such as patrol assignment use normal NDNSF selection.]

[Earlier material in this topic; Origin PDF p. 40, 41, 42: UAV applications are a strong test case for NDNSF because they combine mobility, real-time control, video streaming, distributed sensing, multi-agent coordination, and safety constraints. A UAV network may include a ground station, several drones, cameras, flight controllers, object-detection services, storage capabilities, and mission-planning components. Each component exposes capabilities that naturally look like services.

For the dissertation, the UAV workload is specifically used to stress provider collaboration under mobile and command-sensitive conditions. A ground station may need to discover multiple drones, select one or more providers for a mission, issue targeted commands to known drones, compensate for a failed mission segment, and retrieve video or recording objects without treating large data as RPC payload. This makes UAV-APP a validation workload for the NDNSF framework: it asks whether provider collaboration, authorization, selection status, and supporting low-latency or large-data mechanisms can work together in an operator-facing service container.

Developing such an application over IP creates several recurring problems:

- Address binding. A drone's identity and its IP address may change as it moves, reconnects, or switches networks.
- Service discovery. A ground station needs to know which drones offer camera, telemetry, flight-control, and mission services.
- Failure recovery. If one drone disconnects or fails a task, the mission should compensate using other drones when possible.

4.2 UAV Services

4.3 Architecture]

These operations cover different framework requirements. Selecting a currently available sensing Provider exercises discovery and ACK-based selection. Addressing an already chosen vehicle exercises invocation associated with that Provider rather than selecting an arbitrary substitute.

Telemetry and mission-status retrieval exercise the association of returned Data with the intended vehicle and active operation. Long-running operations supply cases for cancellation, deadlines, and handling late status. Tests must distinguish rejection of an invalid command from the flight controller's physical execution of an accepted command.

Live video and recordings exercise different data paths. Camera Start/Stop is a service-control operation; the resulting live stream uses the named-data streaming interface. Recording references and complete stored objects require exact object retrieval rather than assumptions about cache persistence. These functions therefore contribute to validating stream lifecycle, ordering and gap handling, protected content delivery, and data-product retrieval. Video codecs, image processing, operator interfaces, and flight safety remain application responsibilities; their use of NDNSF is nevertheless part of framework validation.

[Old; Origin PDF p. 41: • Large data. Live video and recorded video should use streaming and segmented data retrieval instead of small RPC payloads.]

[Earlier material in this topic; Origin PDF p. 40: 4.1 Why UAV Networks Need a Service Framework

• NAT and mobility. Direct connectivity to drones is not always available, especially in ad hoc or field deployments.]

4.2 Operator Workflow and State Interpretation

[Old; Origin PDF p. 42: 4.4 Operator Workflow and State Discipline]

A typical session starts with an operator discovering sensing services and choosing a vehicle. Subsequent camera or mission operations retain that vehicle identity unless the operator or a stated mission policy authorizes replacement. The interface associates status with the vehicle and operation that produced it. A delayed observation must not be presented as evidence that a newly issued command succeeded. Observation time, operation identity, and the application's stale-data threshold are therefore part of the validation contract.

[Earlier material in this topic; Origin PDF p. 43: Table 8: Operator-facing state discipline planned for NDNSF-UAV-APP. State Area Planned Behavior Vehicle selection The GS keeps the operator's selected vehicle stable while telemetry, video, and service updates arrive asynchronously. Readiness The GS presents an integrated readiness view for camera, flight controller, telemetry, mission, video, recording, and storage-related capabilities. Telemetry freshness The GS distinguishes current, stale, and missing telemetry so that old data is not mistaken for current state. Command

lifecycle Control and mission commands expose observable progress, completion, and timeout outcomes instead of opaque success/failure messages. Mission progress Patrol missions expose assignment, progress, partial completion, cancellation, and compensation behavior across drones. Failure behavior Timeouts and device/link failures surface diagnostic information so that the operator or application policy can decide the next action.]

The current UAV experiment assumes a camera fixed to the vehicle body. This is an experimental configuration that simplifies geometric calibration; it is not a requirement imposed by NDNSF. Vehicle-attitude disturbance and camera-to-body calibration therefore remain application-level validation concerns.

The proposed evaluation follows a complete session rather than testing isolated buttons: obtain status, start a camera stream, inspect incremental frames, stop the stream, find a recording, and retrieve the selected recorded object. Camera control uses service invocation; frame delivery and recording retrieval use their respective data paths. An implementation may record while no live viewer is attached, so recording availability should not be inferred from stream subscription. The test records whether the configured application supports that mode and whether recording references identify the correct producer and version.

[Old; Origin PDF p. 42: The UAV application is not intended as a replacement for an existing ground control station. Its purpose is to validate whether NDNSF can support a real UAV service workload. For that validation to be meaningful, the GS must still answer the operator questions that matter in practice: which vehicle is selected, whether it is ready and safe to control, which command or mission is active, whether telemetry and video are fresh, and what will happen if a link or device fails. The proposed UAV work therefore treats the GS as a state-driven operator console, not only as a collection of service invocations.]

4.3 Live Video, Recording, and Perception Services

[Old; Origin PDF p. 43: 4.5 Live Video and Recording]

Live viewing and recorded retrieval have different objectives. The live path prioritizes recent usable frames within a latency budget; delayed frames may no longer be useful to the operator. A recording path instead needs a defined retention interval and identifiable segments for later retrieval.

Stream/session identity, producer identity, and frame or segment sequence

[Old; Origin PDF p. 43, 44: Live video is initiated by a GS command to a drone-specific video-control service. The drone negotiates the accepted bitrate and frame size, encodes camera frames

as a streaming codec, and publishes stream Data under names that include a stream identifier and monotonically increasing packet sequence. Each packet also carries stream metadata so the receiver can reject stale packets from a previous stream. The GS fetches and decodes video with an adaptive low-latency policy that uses observed RTT, bitrate, timeout pressure, decoder backlog, and published/decoded gap to tune the prefetch window and stale-frame skipping policy.

Recording is intentionally different from live control. The drone may keep its camera active and record locally even when live streaming is off. The recorded object is represented as a large-data reference to signed and encrypted Data segments. The ground station discovers available recordings through an authorized service invocation, then fetches the referenced segments by name and decrypts locally. Thus storage does not need to understand service authorization; authorization and key material are handled by NDN SF service messages, while replay or stale-data problems are handled by session identifiers, stream identifiers, and signed Data names.]

Table 8: UAV workflow contracts to exercise in application validation.

Operation	Application meaning	Observable framework outcome
Telemetry and status	Observation of the selected vehicle	Correct producer and operation association; stale data identified
Camera Start/Stop	Change the selected camera’s activity	Authorized invocation and terminal status distinct from frame arrival
Live video	Recent frames from the requested stream	Start delay, useful frame delivery, gaps, bounded buffering
Recording retrieval	Obtain a particular stored capture	Exact object and producer validated; Content access enforced
Mission operation	Issue bounded work and follow progress	Admission, progress, timeout, and completion distinguished

must prevent a late frame from an earlier session from appearing in the current one. These are application validation requirements, not claims that every delivery policy is implemented.

The test should vary frame size, production rate, available bandwidth, and receiver buffering independently. Reducing resolution or dropping obsolete frames may reduce viewing delay but also reduce visual information; it is not an unconditional networking improvement. Report delivered frame rate, age of displayed frames, gaps, and recording completeness alongside service-call latency. Authorization to start recording, access a stream, and read a retained object must be checked at the corresponding operation or object boundary.

Perception adds computation to acquisition: a sensing service supplies a named image, an analysis service obtains the permitted image and runs a fixed detector, and the requester retrieves

annotations referring to that image. Telemetry, camera control, recording, flight commands, and analysis therefore exercise different framework paths. Multi-view collaboration combines those paths rather than replacing the rest of the UAV application.

4.4 Flight Commands and Local Safety

[Old; Origin PDF p. 44: 4.6 Flight Control and Safety]

Three responsibilities must remain distinct: the NDNSF runtime checks service authorization and request state; the UAV application manages operator authority and mission state; the vehicle and its safety logic determine whether an operation is safe.

The evaluation will examine where vehicle-specific operator authority is checked, in addition to service-invocation permission. The current ground station checks operator leases; the drone-side execution handler does not independently repeat that check. End-to-end tests will assess whether expired or unauthorized operator commands are rejected along the command path.

Flight-related services connect the application to the vehicle's flight-control interface, such as MAVLink and PX4 [25, 26]. Authorization to submit a command is separate from its operational safety. Geofencing, vehicle readiness, collision avoidance, and local failsafe behavior belong to the vehicle/application safety design. The networking study should use bounded supervisory commands and an explicit loss-of-contact policy, rather than assume that a service invocation is suitable for every low-level flight-control loop.

For a bounded navigation or camera-pointing operation, validation should record vehicle readiness, command acceptance, observed progress, and the final application outcome separately. Readiness checks include a current autopilot heartbeat, a suitable position estimate, battery condition, and the permitted flight mode. The flight controller's command acknowledgment and observed vehicle state must be distinguished from NDNSF ACK Data. A positive NDNSF ACK cannot establish that the vehicle reached a waypoint. Late status from an old command must not complete a newer one.

[Old; Origin PDF p. 44: Flight-control commands are naturally targeted at a specific drone. The proposed work uses Targeted NDNSF invocation for commands such as arm, takeoff, land, goto,

emergency stop, and manual control. The drone-side adapter supports mock mode for MiniNDN tests and MAVLink/PX4 mode for SITL or real flight controllers [22, 23]. Manual-control commands are continuous control messages rather than mission services; their responses can carry telemetry so the GS can update altitude, battery, GPS, speed, controller state, and flight-safety indicators while the operator is flying.

Safety mechanisms include:

- readiness gates based on heartbeat, GPS/EKF, battery, mode, and arming checks;
- command acknowledgments from the flight controller;]

Timeout and reconnection tests must verify a locally configured hold, return, or landing policy appropriate to the vehicle and operating conditions. Continuous manual control requires a separate control-rate and stale-command safety assessment; it is not assumed safe merely because supervisory calls work. In the fixed-camera configuration, interpreting a view also requires time-aligned vehicle attitude and camera-to-body calibration. Service-message delivery alone does not establish flight safety or geometric accuracy.

[Old; Origin PDF p. 44: • manual-control timeouts that stop sending stale control values;

- emergency stop or hold/land behavior after communication failure;
- explicit warnings that real propeller tests require a validated safety procedure.]

4.5 Mission Assignment and Partial Failure

Mission progress, reassignment policy, and compensation remain application responsibilities. The NDNSF runtime carries the associated service exchanges; it does not interpret a vehicle's mission outcome or provide atomic rollback of physical actions.

Multi-vehicle patrol provides a secondary workload: an application divides a region into tasks, tracks their completion, and decides whether unfinished tasks can be reassigned. Its relevance is the association of task state and returned observations with the active assignment. This is broader than multi-view analysis, but it should not become an additional unbounded algorithm-development requirement. The primary collaboration experiment below fixes its task and analysis method so that network and protocol effects can be isolated.

[Old; Origin PDF p. 45: The proposed patrol service demonstrates provider collaboration. A ground station creates a waypoint mission and partitions it among two or more drones. The design clusters waypoints according to the number and position of available drones so each drone receives a compact region and returns to its own departure point after completing its segment. Each drone

receives a mission segment, executes it through MAVLink, and returns status. If a drone fails or returns an incomplete result, the GS can issue a compensation request for the missing segment rather than restarting the entire mission.]

[Earlier material in this topic; Origin PDF p. 45: 4.7 Multi-UAV Patrol]

For both workflows, failure experiments should include a missing observation, a disconnected selected UAV, and delayed status after a replacement decision. Report the time during which a feasible Provider is actually reachable as well as the request deadline. Retransmission and synchronization cannot manufacture radio coverage or extend an expired application time budget. A partial completion rate can therefore reflect availability limits even when retrieval recovery behaves correctly.

[Old; Origin PDF p. 45: This design illustrates a key NDNSF principle: compensation requests are application-level parts of the same logical mission, while NDNSF provides the secure service invocation, provider selection, and message verification substrate.]

4.6 Representative Collaboration: Multi-View Collection and Analysis

An operator chooses a target location or area on a map. Request Data carries only the service-protected descriptor needed to assess coverage, viewing feasibility, sensors, and timing. Eligible UAVs return positive ACKs. The proposed workflow assigns complementary viewing roles and a processing role, then delivers execution details and appropriate keys through Selection.

Each sensing UAV signs encrypted ViewImage Data under its own authorized namespace, with observation time and available pose/calibration metadata. The compute Provider retrieves permitted images, performs the specified analysis, and publishes signed Annotation Data. Its encrypted result supplies original image references and permitted image keys to the requester, which retrieves the images and renders annotations. The workflow reuses the original signed images rather than republishing them under another producer's identity.

The Global Positioning System (GPS) alone does not establish camera visibility, depth, or correspondence between views. Improved recognition or localization requires ground truth, a single-view control, and a specified fusion method. MiniNDN [27] and PX4 Software in the Loop (SITL) [26] can exercise network behavior and application/autopilot integration respectively; nei-

ther establishes real-flight safety.

[Earlier material in this topic; Origin PDF p. 45: The current UAV-APP is a research prototype, not yet a field-ready or safety-certified ground-control system. This distinction is important for the proposal: practical use, SITL, and selected real-machine testing may expose new requirements or changes in command handling, telemetry freshness, video behavior, safety policy, deployment, or operator workflow. Those discoveries are part of the research value because they can feed back into NDNSF's framework mechanisms.]

Chapter 5

Distributed Inference over NDNSF

[Old; Origin PDF p. 47: Distributed AI Inference over NDNSF]

NDNSF-DI is the distributed-inference (DI) application over NDNSF. It exercises discovery, authorization, model and tensor exchange, and execution management through single-Provider inference and collaborative model execution. Its computation and retained state provide a contrasting workload to UAV sensing and external actions. Chapter 6 evaluates the shared framework mechanisms and application-specific checks.

[Earlier material in this topic; Origin PDF p. 47, 48, 49: 5.2 Approaches to Distributed AI Inference

5.3 NDNSF-Based Inference Architecture

- input and output contracts that describe tensor names, shapes, dtypes, and codecs;

ONNX graph Model operators, tensors, and edges Tensor dependency DAG Producer-consumer relationships among intermediate tensors Candidate split points Boundaries ranked by activation size and execution constraints Chunk graph Executable chunks and cross-chunk tensor dependencies NDNSF-DI collaboration plan Provider roles, artifact references, object names, and fallback policy Figure 5: Automatic generation path from an ONNX computation graph to an NDNSF-DistributedInference collaboration plan.]

5.1 Application Structure and Inference Workflows

NDNSF-DI provides an inference application layer over the framework. It interprets model and task descriptions, coordinates model preparation and execution, and exposes inference results to callers. Its native requester and Provider interfaces include request submission, service registration, result/event handling, and cancellation; conversation-related interfaces also carry continuation state. The presence of an interface is not evidence that every model or recovery configuration has passed end-to-end tests.

[Earlier material in this topic; Origin PDF p. 47, 48: In IP-based systems, distributed inference often relies on custom orchestration: host addresses for model stages, separate object stores for model artifacts, custom integrity checks, ad hoc retry behavior, and manual placement policies.

This makes it hard to build applications that remain correct when providers join, leave, fail, or move.

- Replication/data parallel inference: multiple providers hold the same model and serve independent requests.

The proposed NDNSF-DistributedInference application treats an inference task as a named service with associated roles, stages, artifacts, tensor dependencies, and runtime requirements. A service policy defines:]

Python bindings adapt the native C++ path rather than maintain a second request state machine.

[Earlier material in this topic; Origin PDF p. 48: • the service name, such as /AI/YOLO/SplitInference;]

The NDNSF runtime supplies reusable operation, completion, and event-reading mechanisms. DI defines model preparation, placement, tensor contracts, K/V compatibility, and the rules for committing a completed turn to conversation state.

A single-Provider inference request exercises service discovery, permission checks, selection, protected input delivery, and result retrieval without requiring model partitioning. Preparing a cold model and reusing a resident model exercise different readiness and artifact-access paths. Model files and recorded artifacts may use managed repository storage; the forwarding Content Store remains opportunistic caching, not guaranteed persistence. DI defines model identity, compatibility, and placement policy, while NDNSF supplies service lifecycle and named-object exchange.

[Old; Origin PDF p. 47: Distributed AI inference is another strong test case for NDNSF. AI models are large, runtime artifacts are sensitive, intermediate tensors can be large, and inference providers may be heterogeneous. A model may be split across edge devices, drones, ground stations, GPUs, and cloud resources. The user wants to invoke an inference service by model or task name, while the system determines where model shards and stages should run.

For the dissertation, distributed inference stresses provider collaboration in a different way from UAV workloads. Providers are not collaborating because they are vehicles in a shared physical mission; they collaborate because a single inference task may require multiple model chunks, runtime artifacts, activation exchanges, and dependency-ordered execution steps. This workload therefore tests whether NDNSF can keep collaboration generic: the framework should coordinate authorized providers, large-data references, status diagnosis, and result verification, while the inference layer supplies the model-specific dependency graph and execution plan.]

[Earlier material in this topic; Origin PDF p. 47, 48: 5.1 Motivation

- Pipeline partitioning: model stages execute sequentially on different providers, following split-computing ideas such as DNN partitioning across mobile, edge, and cloud resources [24,25].

- model stages, chunks, and roles;]

Streamed inference and conversation workflows extend coverage to incremental outputs, cancellation, terminal status, and request-specific state. Model weights, activation tensors, and key/value (K/V) state have distinct lifetimes. The model computes K/V tensors and the inference runtime retains them for later decoding; shared buffers can avoid copying the whole cache at each step [28]. Resident weights do not replace this state. DI owns tokenization and KV compatibility rules; framework validation concerns the authorized request, data exchange, and lifecycle through which the application uses that state. The evaluation must separate the benefits of local K/V reuse from those of NDNFSF’s communication and execution mechanisms.

[Earlier material in this topic; Origin PDF p. 47, 48, 49, 50: The dissertation will review the main distributed inference approaches:

- Edge-cloud split inference: lightweight preprocessing happens at the edge while heavier stages run on ground-station or cloud nodes.

This proposal uses two split terms to avoid ambiguity. A stage split divides the model into sequential stages, such as backbone, head, and merge tasks. This is the natural first choice for large language models and many pipeline-style models because each provider can own a contiguous part of the execution. A shard split divides work inside one stage across multiple providers. Sharding can help when no single provider can fit or execute a stage, but it usually adds communication and merge overhead. The planner should therefore prefer reusable stage layouts and introduce shard layouts only when provider memory or compute capacity requires them.

- dependency edges that state which role produces each intermediate tensor bundle and which roles consume it;

The user should not need to manually create a plan for every request. Instead, the service controller or model-splitting tool should generate the plan when the model is prepared. For ONNX models, which provide a standard computation graph representation for machine-learning models [27], the proposed path is shown in Figure 5.

The same policy interface should also support non-ONNX models by accepting a model-specific splitter output. At runtime, the user invokes the service name; providers assigned to roles fetch the artifacts associated with their roles, prefetch predictable input tensor bundles, execute their model chunk, publish output tensor bundles, and return or forward the final prediction.]

5.2 Representative Collaboration: Multi-Provider Inference

A planner uses the prepared model and validated ACK offers to choose a partition and Provider assignment after candidate collection closes. Selected Providers prepare or reuse their assigned resources after authenticating Selection. Pipeline partitioning assigns sequential layers or layer

groups. Tensor partitioning divides operations or tensors and requires the corresponding collective exchanges. Providers exchange activations or collective inputs over declared dependencies. Open Neural Network Exchange (ONNX) [29] describes the computation graph; export alone does not implement cross-Provider collectives or establish numerical correctness. Model partitioning has substantial prior work [30,31]; the NDNSF question concerns runtime assignment and authorized named-data execution.

[Earlier material in this topic; Origin PDF p. 48, 49: • Layer or operator partitioning: the model graph is split at finer granularity according to compute and memory constraints.

- Collaborative inference: multiple providers cross-check, aggregate, or vote on results.
- required artifacts such as ONNX chunks, PyTorch modules, or runtime artifacts;
- candidate providers, provider roles, and fallback policy.]

Autoregressive generation also uses each generated token in a later decoding step. This temporal dependency differs from forward computation through the layers. The evaluation separates model preparation, prefill, time to first token, subsequent decoding, state storage, and data transfer. Claims remain specific to tested models, precisions, partitions, and devices. A feasible single-Provider baseline is retained; if a model cannot fit one device, distributed feasibility is meaningful but a speedup against a non-running baseline is not.

[Earlier material in this topic; Origin PDF p. 48: • Early-exit inference: computation can stop early when a confidence threshold is reached [26].]

5.3 From a Model Graph to Runtime Assignments

Local preparation verifies a reusable model package, its source identity, and supported interfaces. In this chapter, PreparedModel denotes that locally validated, reusable model package and session state; it is not a Provider assignment, resource reservation, or proof that a remote runner is ready.

After ACK collection closes, the native planner validates offers, enumerates supported partitions, and validates placement. A reusable or manually authored template may constrain that search; it does not preselect the runtime Providers.

The planner obtains artifact references and seals the execution plan before committing it through the NDNSF runtime. After authenticating Selection, each selected Provider obtains, as-

sembles, or reuses its permitted resources before execution.

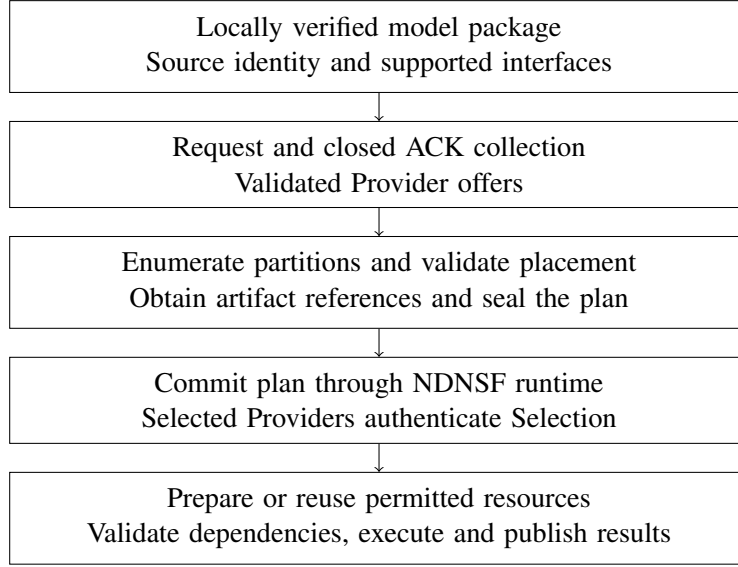


Figure 3: Model preparation and request-specific planning in the native path. Arrows show processing order; model, device, and recovery coverage still require separate validation.

Each fragment contract needs defined input and output tensor names, shapes or supported dynamic-shape constraints, element types, serialization, and model-version compatibility. A split across a skip connection or a branching graph may have multiple live outputs, not a single vector. The receiving fragment must obtain all required inputs before execution. A useful negative test supplies a correctly signed tensor with the wrong shape or model identity: protocol provenance may be valid while the application contract is not.

Pipeline partitioning assigns consecutive computation stages and transfers their boundary tensors. Tensor parallelism splits an operation across participants and adds exchanges such as reductions or gathers. Model replication assigns independent work to complete copies. These are different execution strategies. Early exit changes how much model computation is performed [32]; combining predictions from several models changes the analysis method. Both must be separated from placement effects. Edge–cloud describes device placement, not a distinct partition operator. ONNX export supplies a graph representation [29]; usable cuts and collective operations still depend on the supported runtime and model operators.

Preparation cost matters when a Provider lacks the required artifact or compatible loaded session. Evaluation should distinguish artifact transfer, session creation, accelerator memory al-

location, and execution. A warm run that reuses resident weights cannot be compared to a cold baseline without reporting that difference. Placement can favor a Provider that already holds compatible state, but that policy must respect request authorization and cannot assume that cached state is transferable to any new request.

5.4 Planning Inputs, Feasibility, and Policy Comparisons

The planning study considers a bounded set of supported partition candidates. It separates feasibility constraints from optimization preferences: an unsupported operator, an incompatible tensor contract, or insufficient memory excludes an assignment; estimated execution or transfer time ranks otherwise feasible assignments. Estimates need a stated source and measurement age. A Provider's positive ACK is a willingness report, not an independently verified hardware benchmark.

[Old; Origin PDF p. 50: A major proposed contribution is automatic generation of collaboration plans from AI model structure and resource information. The input includes:

- model graph, tensor dependency DAG, candidate cut points, and estimated activation sizes;
- provider compute, memory, accelerator, and battery state, with a homogeneous-provider default when detailed profiles are unavailable;
- network latency, RTT, and throughput estimates;
- artifact placement and cache state;
- security and permission constraints.]

[Earlier material in this topic; Origin PDF p. 50: 5.4 Automatic Collaboration Plan Generation]

Compare a fixed valid mapping, a simple resource-aware mapping, and any implemented adaptive policy on the same Provider candidates and supported partitions. Record planning time, per-Provider utilization, and end-to-end cost. Where a policy predicts completion time,

[Old; Origin PDF p. 50: The output is an NDNSF service plan that assigns model roles and chunks to providers, identifies required artifacts and tensor bundles, records predictable object names for prefetch, and defines fallback behavior.]

Table 9: Inputs to the proposed DI planning comparison. These are evaluation dimensions, not a claim of a completed general optimizer.

Planning input	Feasibility / cost question	Experimental control
Model and partition contract	Operators, precision, tensor interfaces and compatible artifacts	Freeze model/export identity and supported cuts
Device resources	Weights, activations, workspace and retained key/value (K/V) state fit	Measure peak memory; do not use weight size alone
Computation and communication	Do capacity or execution-time benefits justify transfer costs?	Profile boundary bytes and per-stage time under the same load
Preparation and residency	Are artifacts, sessions and compatible state already present?	Separate cold preparation from warm execution
Permission and availability	Can each assigned role obtain its required objects before the deadline?	Apply authorization and deadline constraints before ranking

compare its prediction with the observed value; a policy using only feasibility rules needs no timing-prediction claim. If no complete feasible assignment exists, report that outcome rather than silently dropping a dependency. The initial question is whether runtime information improves assignment under stated conditions. Model splitting, Provider selection, and data-dependency enforcement remain independently testable.

5.5 Tensor Exchange and Autoregressive State

For a pipeline, an upstream fragment produces an activation tensor, makes it available as protected named Data, and supplies its reference through the declared dependency. The downstream fragment retrieves and validates that object, checks the tensor contract, and performs its assigned computation. A fan-out reuses an upstream output for several authorized consumers. A fan-in waits for the set of inputs required by the computation. The collaboration protocol decides which producers and edges are permitted; DI decides whether the tensor values and layouts are suitable for the model. [Old; Origin PDF p. 50: The planned runtime should hide dependency management from AI application developers. A provider should not contain YOLO-specific or fully-connected network-specific communication code. Instead, each provider role reads its local input edges from the policy, prefetches all predictable input objects, merges tensor bundles, executes its assigned ONNX or

model-specific chunk, and publishes the tensor bundles required by its output edges. This executor model is intended to support chain, fan-in, and fan-out graphs. YOLO 2x2 and small fully connected examples are therefore validation cases for the same dependency-driven execution design rather than separate hand-written systems.]

[Earlier material in this topic; Origin PDF p. 50: 5.5 Dependency-Driven Execution]

Autoregressive inference adds a dependency across decoding steps. When K/V caching is enabled, computation after prefill uses the preceding token and retained K/V state instead of recomputing the full prefix. With layer-based partitioning, each participating runtime can retain the K/V tensors for its assigned layers. A generated token or the information needed to select it returns to the component driving the next decoding step. Consequently, the forward path within one evaluation can follow a directed acyclic graph while the whole generation process repeatedly traverses that graph over time.

Keeping weights in accelerator memory does not eliminate the logical K/V inputs. A runtime may reuse device buffers and bind them directly rather than serialize and transmit the entire cache on each step [28]. Reassignment or conversation continuation needs an explicit compatibility and state-transfer policy; silently starting a replacement with unrelated K/V state is not valid recovery. The evaluation separates savings from local K/V reuse, remote transfer costs, and framework orchestration costs.

The initial numerical validation should compare a supported partitioned execution with an unsplit execution of the same model, precision, inputs, and decoding policy. Appropriate tensor tolerances or deterministic output comparisons must be specified before measurement. A small synthetic model can validate a tensor route, but cannot establish memory feasibility, throughput, or recovery for a large pretrained model. These controls make it possible to distinguish a working communication path from a correct and useful distributed-inference application.

We are preparing multi-node GPU experiments on TigerCluster, the external multi-GPU cluster used for this evaluation, to evaluate NDNFS-DI. Providers will run on separate GPU nodes to check inference correctness and measure end-to-end latency, inter-Provider data-transfer overhead, and peak GPU memory use. These cluster experiments will complement, rather than replace, UAV wireless and mobility tests.

Chapter 6

Research and Evaluation Plan

[Old; Origin PDF p. 51: 5.7 Prototype and Evaluation Plan]

6.1 Cross-Application Validation

The study uses UAV and DI workflows to examine which discovery, authorization, dependency, and lifecycle rules can be shared, and which decisions require application-specific contracts. End-to-end measurements follow each workload through its invocation and named-object exchange paths. Single-Provider operations examine RQ1 and RQ3; workflows with complementary roles also examine RQ2.

[Earlier material in this topic; Origin PDF p. 52: • single-provider layouts versus shared-backbone and multi-provider layouts;]

The proposed division of responsibilities is explicit: the framework checks who may produce an object and where it may be used; the application checks whether its content is useful and correct. Table 10 makes that boundary testable. The NDNFSF runtime may invoke application validators through a common interface; this does not make their domain knowledge an NDNFSF protocol rule.

Table 10: Proposed shared checks and application responsibilities, to be tested in both applications.

Shared framework check	UAV contract	DI contract
Validate signer, assigned role, and dependency edge	Define required view, observation time and admissible sensor metadata	Define model/partition identity and tensor dtype/shape compatibility
Validate complete named objects and explicit reuse permission	Decide whether an existing image still satisfies coverage and timing	Decide whether artifacts or key/value (K/V) state match the model and continuation
Enforce local cancellation/replacement state before dependent work	Stop or compensate physical actions; report what actually occurred	Drain computation, release resources, and commit or discard continuation state
Authenticate provenance, not semantic truth	Validate visibility, localization and recognition against observations	Validate numerical outputs against a reference execution

For example, a correctly signed image from the assigned UAV can still show the wrong area. A correctly signed tensor can still contain wrong values. The framework must not certify either as a correct application result. The research deliverable is a shared set of object-acceptance and execution-state rules, with explicit points for application checks. Both applications will test invalid cases and confirm that valid work still completes. Control, telemetry, streaming, recording, preparation, and single-Provider inference remain part of this study; collaboration adds tests rather than replacing that coverage.

Table 11 groups the planned application experiments by the framework mechanisms they exercise.

Table 11: Application workflows and intended NDNSF validation coverage.

Application workflow	Framework mechanism	Required observation
UAV control/telemetry; DI single-Provider inference	Discovery, selection, service authorization	Correct Provider/request association; unauthorized invocation rejected
UAV live video; DI incremental output	Stream or invocation-output lifecycle	Validated outputs, termination, cancellation, and defined gap handling
UAV recordings; DI model/input objects	Named-object references and retrieval	Exact permitted object reconstructed; tampered or wrong object rejected
UAV mission status; DI preparation/conversation	Operation and request lifecycle	Correct status ordering; late actions cannot revive closed work
UAV multi-view; DI partitioned inference	Role assignment and named dependencies	Assigned producers only; missing dependencies and replacement handled
Permission changes in either application	Controller status and invocation checks	Grant discovered/applied; withdrawn permission rejected after update

For each evaluated workflow, the study will document the application function, NDNSF API path, configuration, expected outcome, and recorded results. Local tests will check individual components; multi-process experiments will evaluate communication and execution across nodes. Image-analysis accuracy, numerical correctness, and vehicle behavior will be assessed with application-specific criteria alongside service completion.

[Earlier material in this topic; Origin PDF p. 45, 52: The UAV application will be evaluated in three stages:

2. SITL tests: PX4/jMAVSim control for arm, takeoff, land, manual control, and mission execution.

- chain execution versus fan-in/fan-out dependency execution;]

The current native implementation has results from tests of individual components and selected multi-process workflows. The planned campaigns will extend this evidence to the selected model, device, and recovery configurations. New tensor/hybrid, multi-accelerator, and large-language-model recovery configurations require their own experiments; the historical small-model measurements provide a separate starting point. Representative workflows will cover the mecha-

nism categories in Table 11, with conclusions limited to the configurations evaluated.

[Earlier material in this topic; Origin PDF p. 51: The prototype will focus on ONNX split inference with both YOLO-style and small fully connected or transformer-style models, then generalize from chain-like examples to fan-in and fan-out dependency graphs. Planned experiments include:]

6.2 Preliminary Evidence and Its Limits

Invocation campaigns below measure discovery, selection, and execution under their recorded configurations; they do not test a multi-role collaboration plan. The earlier DI diagnostic is separate application evidence, not proof that the initial framework supported role-aware collaboration.

[Earlier material in this topic; Origin PDF p. 51: 5.6 Current Prototype Status]

The following numbers are retained from recorded campaigns, not new experiments in this revision [33]. Comparative uncertainty is estimated across runs with independent seeds, not by treating requests within one run as independent observations. Here, CI denotes confidence interval. Latency calculated only from successful requests must be reported alongside completion and time-out rates. A baseline label identifies the tested implementation and configuration, not every feature of the named framework.

[Old; Origin PDF p. 34: The completed NDN SF evaluation compares NDN SF against NSC and gRPC in MiniNDN. The evaluation focuses on service latency and success rate under wired, lossy, intermittent-provider, wireless mobility, adaptive-admission, selective ACK, and provider-selection scenarios. The key result is that NDN SF can outperform NSC and improve reliability under provider diversity and intermittent availability, while maintaining reasonable latency relative to IP-based RPC in lossless conditions.]

[Earlier material in this topic; Origin PDF p. 34: 3.8 Completed Foundation: NDN SF v0.1 Evaluation]

In the work-efficiency comparison, NSC-4 also completed 1,800/1,800 requests, with 1.002 execution starts per request (rounded). This does not establish a work-efficiency advantage over NSC. The mobility comparison uses common traces, publication times, deadlines, and four Providers per system. The switching holdout uses seeds 72–81 and a registered reachability-based classification. Of its 1,312 aligned requests per system, NDN SF completed 1,311, gRPC 1,297, and NSC 1,296. For each seed, p95 is the 95th-percentile latency calculated from that system’s successful requests, so the latency comparison is affected by different failure sets. Overall cover-

age controls using seeds 62–71 are a separate campaign, not the parent sample of this holdout. The small-model diagnostic is retained from earlier records; its raw-run/model provenance remains to

be rechecked. Communication/setup overhead is a possible explanation,

[Earlier material in this topic; Origin PDF p. 51: The current prototype has moved from deterministic trace execution toward real ONNX execution over the full MiniNDN network harness. The latest NativeTracer campaign uses a small Qwen-style model artifact and runs the same Backbone/Head/Merge workflow through real model execution, repo-backed artifact fetching, service manifests, and NDNSF large-data references. Providers execute assigned roles from the generated plan and exchange named intermediate tensor objects instead of embedding large tensors in service payloads.]

The current layout campaign compares two executable runtime layouts under the same full-network harness. In ten successful runs per layout, the shared-backbone layout had mean latency 259.4 ms, p50 267.7 ms, p95 283.9 ms, and standard deviation 22.1 ms. The single-provider layout had mean latency 186.8 ms, p50 191.7 ms, p95 206.3 ms, and standard deviation 20.0 ms. For this small model, single-provider execution is faster because it avoids the fixed cost of cross-provider dependency exchange. This is a useful negative result: NDNSF-DI should not split a model merely because multiple providers are available; the planner must compare compute cost, intermediate-data size, provider capacity, RTT, bandwidth, and artifact placement before selecting a layout.]

Table 12: Selected evidence and limits; values are unchanged from their records.

Campaign	Observation	Supported interpretation / limit
Ten-seed switching holdout	1,312 matched switching requests; NDNSF minus gRPC seed-p95 difference: -597 ms, 95% CI $[-992, -200]$; versus NSC: -2618 ms, CI $[-2916, -2315]$	Lower conditional tail latency in the tested four-Provider configuration, not universal mobility or success-rate superiority
Six-seed work-efficiency repeat	NDNSF: 1,798/1,800 completed, 1,800 execution starts; gRPC-PAR-4: 1,800/1,800, 7,180 starts	Fewer starts than this parallel policy; starts are not messages, bytes, CPU time, or energy
Earlier small-model DI diagnostic	Ten successful runs/layout: mean 259.4 ms distributed versus 186.8 ms single Provider; p95 283.9 versus 206.3 ms	Splitting was slower for this model/configuration; small success-conditioned sample does not establish a general tail-latency result

[Earlier material in this topic; Origin PDF p. 35: Table 5: Selected reliability results summarized from the submitted NDNSF v0.1 evaluation. Scenario Condition NSC gRPC NDNSF Intermittent wired providers 10% failure probability 83.25% 83.35% 99.82% Intermittent wired providers 20% failure probability 66.58% 66.69% 99.46% Intermittent wired providers 30% failure probability 49.92% 65.36% 96.70% Wireless mobile providers 100 m AP range 45.42% 45.23% 96.25% Wireless mobile providers 150 m AP range 68.75% 68.05% 100.00% Wireless mobile providers 200 m AP range 88.33% 87.14% 100.00%]

not an established cause.

The transition experiment reports 357/357 completions for NDNSF without pre-registering Provider addresses, 0/357 for static-list baselines, and 357/357 for pre-registered baseline variants. It isolates the discovery configuration. A gRPC resolver or other discovery integration can update a candidate list [34]; the result does not prove that gRPC or NSC cannot discover a new Provider. [Earlier material in this topic; Origin PDF p. 51: Earlier YOLO-style and small fully connected examples remain useful validation cases for dependency-driven execution, but the remaining research work is now to make the planner and runtime more general: remove model-specific assumptions from provider logic, use ONNX tensor DAGs and provider/network profiles to generate reusable plans, and evaluate when multi-provider execution helps rather than only adding coordination overhead.]

Historical v0.1 loss results include 599/600 completions at 1% per-link loss, 598/600 at 3%, and a best recorded 539/600 at 10%. The last is a selected historical run, not a current aggregate estimate or a matched comparison against later baseline data. The expression $1 - (1 - l)^4$ assumes four independent packet traversals with equal loss l ; it is not the full service-failure probability after retries. Timeouts under heavy loss do not, without further interventions, identify one exclusive cause or rule out protocol/policy effects.

[Old; Origin PDF p. 34, 35: The strongest evidence comes from scenarios where endpoint binding or a single provider becomes fragile. In the lossy wired Memphis–CSU–UCLA path, packet round-trip loss is $1 - (1 - l)^4$ for per-link loss l . NDNSF completed 599 of 600 requests at 1% link loss, where the packet round-trip loss is 3.94%, and 598 of 600 requests at 3% link loss, where the packet round-trip loss is 11.47%. At 10% link loss, where packet round-trip loss rises to 34.39%, the best measured NDNSF result was 539 of 600 requests. This identifies the bounded-time delivery limit of SVS-based service invocation under heavy loss, rather than a failure of the NDNSF service model. In a wired multi-provider experiment with intermittent provider rejection, NDNSF maintained 99.46% success at 20% provider-failure probability and 96.70% success at 30%, whereas the single-provider baselines dropped to roughly two-thirds success or lower. In the wireless mobility experiment, NDNSF achieved 96.25% success at 100 m AP range and 100% at larger ranges, while NSC and gRPC were much lower when bound to one mobile provider. The adaptive-admission and Selective ACK experiments further show why the framework needs runtime metadata: admission control kept P95 latency below 270 ms under overload, and Selective ACK raised success from 1.82% to 100% when the nearest provider was overloaded. Together, these results support the v0.1 claim that one-to-many invocation, provider metadata, and explicit selection improve reliability without giving up data-centric security.]

Prototype admission, ACK-policy, and availability observations are preserved in Appendix A. Their raw-run provenance or comparison controls require further verification, so they motivate experiments rather than establish the present comparative conclusions.

6.3 Evidence Needed for Each Research Question

Table 13: Planned comparisons, correctness tests, and measurements for each research question.

RQ	Comparison	Correctness / failure observation	Costs and outcomes
RQ1	ABE-backed authorization; DNMP-inspired role rules plus content encryption (Section 2.4.2); suitable group-key combinations	Unauthorized call never starts handler; replay, wrong challenge, grants, withdrawn/expired permissions, status delay	Completion, discovery delay, crypto work, credential/policy size, grant and withdrawal traffic
RQ2	Same task graph with static/dynamic assignment; controlled removal of each admission check	Wrong role, edge, name, key or execution; same checks for producer/cache Data; incomplete objects; cancellation or reassignment	End-to-end completion, validation cost, intermediate bytes, redundant work, bounded recovery
RQ3	Matched Provider counts, tasks, capacities, deadlines and failure schedules; feasible single-device control	Include failed and timed-out requests; distinguish network, admission, execution and result-retrieval failures	Completion and latency together; cold/warm costs; per-run uncertainty; conditions with no benefit

[Old; Origin PDF p. 55: Table 9: Evaluation matrix for the dissertation. RQ Dissertation Component Evaluation Target Evaluation Methodology RQ1 Literature review NDN service framework gap Related-work comparison of RPC, robotics middleware, pub/sub, workflow systems, NDN service approaches, UAV networking, and distributed inference systems, summarized as framework requirements. RQ2, RQ3 NDN core Security, selection, targeted invocation, local composition, collaboration, and large-data support MiniNDN regression and performance tests for permissions, NAC-ABE token routing, replay protection, provider selection, collaboration status, Targeted invocation, local invocation, and large-data references. RQ4 UAV workload Mobility, freshness, command lifecycle, media objects, and multi-UAV missions GS/Drone MiniNDN tests, SITL MAVLink checks, command lifecycle logs, video stability, telemetry freshness, recording retrieval, mission progress, and failure recovery. RQ5 DI workload Model/data retrieval, split inference, dependency execution, planner decisions, and multi-provider collaboration MiniNDN cluster tests for local-versus-distributed output equality, ONNX graph summary, generated policy checks, reference-based artifact provisioning, activation exchange, layout campaigns, provider utilization, and provider-failure cases. RQ2, RQ4, RQ5 Operations Reproducible deployment and selected real-machine validation Certificate and route procedures, configuration checks, reproducible experiment setup, and selected real-machine tests.]

Each campaign will record source, dependency, and model versions, configurations, seeds, measurement intervals, raw results, and analysis scripts. Scaling experiments will vary candidates and services for discovery/authorization, and dependency edges and object sizes for collaboration.

Ablations will disable individual checks only in controlled tests to isolate their effects. Failed attempts will be retained and classified by their first observed failure; runs that fail before request measurement begins will be reported separately from measured invocation outcomes.

6.4 Application Experiments and Interpretation

Individual invocations or application sessions supply observations. Comparative uncertainty is estimated across independently repeated runs, with shared seeds or traces paired across systems; requests within one run are not assumed independent.

Before each comparative campaign, the workload definition will specify the primary outcome, completion deadline, and criteria for a practically useful improvement. Pilot runs will inform repetition counts; these choices will be fixed before the comparison and reported alongside effect estimates and uncertainty.

To operationalize RQ1, the study defines the following authorization cost model before selecting comparison configurations.

The first RQ1 comparison will retain ABE-encrypted discovery in both designs and change the invocation-authorization mechanism: protected challenges versus DNMP-inspired role-constrained signing. Both retain signature validation, Selection, request-state checks, current service-permission checks, and recipient-scoped input/result protection. This isolates authorization choice from discovery encryption; it does not measure the original DNMP implementation. Service-group encryption is a separate sensitivity comparison. Initial authorization tests use one User, four candidate Providers, and one permitted and one unpermitted service before scaling the configuration.

[Earlier material in this topic; Origin PDF p. 52: • cold-start artifact fetching versus cached artifacts;]

The matched alternatives will use the same permission matrix, discovery protection, and status-validity bound. Shared discovery costs remain in the total but are separated from incremental invocation-authorization costs. Credential and policy model. Consider one Controller domain, one ABE key generation, and n authorized Users. Let k and r denote the mean service permissions and

role certificates per User. Each User retains one identity-signing certificate.

[Earlier material in this topic; Origin PDF p. 45: 1. MiniNDN regression tests: controller, one or more drones, operator console behavior, video start/stop, encrypted recording playback, and patrol service execution.

3. Real-machine validation: one PC and one or more drones with NFD routes, certificates, camera devices, and MAVLink connections.]

The role alternative adds nr role certificates; ABE-backed authorization adds n policy DKEYs. Provider population and permissions remain fixed, and both designs retain the Provider DKEYs needed for discovery.

[Earlier material in this topic; Origin PDF p. 52: • planner decisions under different provider memory, compute, RTT, and bandwidth profiles;

• provider failure and compensation;]

Per-service roles ($r = k$) add $O(nk)$ certificates; one composite role per User adds $O(n)$. With fixed k , both grow linearly with n . These counts describe the chosen credential arrangements, not limits imposed by Trust Schema or DNMP.

Certificate counts are not private-key or NDN packet counts. Shared roles share permission definitions, not private keys. Policy information remains: explicit DKEY service lists contain nk entries in total.

[Earlier material in this topic; Origin PDF p. 52: • manually written policy versus automatically generated policy;]

We will measure key sizes, validation rules, and distribution traffic alongside object counts. NAC motivates this distinction [35]; its CP-ABE analysis does not establish NDNSF's KP-ABE authorization cost. Lifecycle cost. For design X and additive metric d , define the cost per submitted logical invocation as

$$\bar{C}_{X,d}(T) = \frac{C_{X,d}^{\text{setup}} + \sum_{p \in \mathcal{P}} C_{X,d}^p(T)}{Q(T)}, \quad \mathcal{P} = \{\text{invoke, grant, withdraw, maint}\}. \quad (1)$$

Here $Q(T)$ counts logical invocations submitted during a common interval T , not > retries. Each C_p is the total cost of that activity during the same interval; maintenance includes background status checks and renewals, without double counting.

CPU time, transmitted bytes, and management operations are measured separately. Setup

and recurring costs are also reported separately; completion, update delay, latency, and memory are not combined into this additive cost.

An ordinary NDN SF grant replaces the target’s DKEY without rotating public parameters. Controller-wide withdrawal rotation can require replacement DKEYs for all remaining authorized entities, including Providers.

The adapted role baseline uses signed status, version checks, and validity intervals. Withdrawing a User’s invocation permission need not require reissuing every role certificate. This life-cycle is a baseline extension, not a claim about DNMP.

[Earlier material in this topic; Origin PDF p. 52: • local baseline inference versus NDN SF split inference;]

Both designs reject known status rollback and stop admitting calls when required status expires. They use the same validity bound and clock-skew assumption; disconnected nodes may lose availability.

NDN FreshnessPeriod and MustBeFresh do not establish authorization validity [2]. Withdrawing a Provider’s permission to read discovery descriptors requires discovery-key updates in both designs; neither can recall disclosed keys or plaintext.

Table 14: Planned RQ1 configurations. Counts are analytical examples, not measurements.

Configuration	Controlled example	Question to test
G1: Per-service roles	100 Users; four disjoint five-service bundles, 25 Users per bundle. One certificate per allowed service: 500 role certificates versus 100 User DKEYs.	Do fewer objects reduce total cost after policy size and cryptographic work are included?
G2: Shared combination roles	Same permissions; one bundle-role certificate per User: 100 role certificates versus 100 DKEYs.	Does ABE retain an advantage when roles already aggregate permissions?
G3: Frequent withdrawals	Repeat G1/G2 with matched grants and withdrawals. Fix invocation rate and vary withdrawal rate; test User invocation and Provider discovery-read withdrawal separately.	When does rekeying outweigh provisioning savings?

The 100-User example counts Users issued credentials, not Users required to invoke services concurrently. Paired sweeps will vary population and permission overlap. Role-update rules,

workloads, cache conditions, and repetitions will be fixed before comparison.

Updates preserve exact permissions. We will compare costs, legitimate-call completion, and rejection of expired or rollback status, withdrawn permissions, and replayed calls. The aim is a conditional tradeoff, not a universal ranking.

For UAV orchestration, the control relays the same signed, encrypted image objects through the User without changing their producer identity; direct exchange uses the assigned producer names. Both use equivalent protection and the same processing code. Image inputs are held fixed for this comparison, so it does not test a recognition benefit from additional views. Numerical tolerances for DI will be declared for the chosen precision before runs, not adjusted to fit the observed errors.

Each run records its workload, software revision, and network configuration. Report submitted, admitted, selected, completed, rejected, timed-out, and cancelled work separately where observable, with explicit denominators and deadlines.

[Earlier material in this topic; Origin PDF p. 52: • one-provider, two-provider, and multi-provider execution;]

Latency distributions identify the included requests and are reported alongside failure rates. A single exploratory run is labeled as such and does not supply a between-run uncertainty estimate. [Earlier material in this topic; Origin PDF p. 52: • impact of artifact placement on startup latency.]

For UAV sessions, record command completion latency, video start delay, useful frame delivery, recording retrieval completion, and status age at the operator. For multi-view work, additionally record the number of usable views, processing completion, and the analysis metric against labeled observations. The framework experiment first holds the recognition algorithm fixed. A separate application study is needed before attributing higher recognition accuracy to additional views.

[Old; Origin PDF p. 46: Metrics will include command latency, video startup time, decoded frame rate, recording retrieval time, mission completion rate, compensation overhead, telemetry freshness, and failure recovery time.]

[Earlier material in this topic; Origin PDF p. 45: 4.8 Evaluation Plan]

For DI, report model preparation, prefill, time to first token, subsequent output rate, total completion, peak device memory, and bytes transferred between Providers. Compare cold and

warm conditions separately. Evaluate a feasible single-Provider case alongside a supported partitioned case. If the distributed deployment exists because the model cannot fit one device, report that capacity benefit directly rather than inventing a speedup over an infeasible execution. Profile communication, serialization, cryptography, and model computation to locate the limiting cost. [Old; Origin PDF p. 52: Metrics will include end-to-end latency, artifact retrieval time, provider utilization, success rate, output consistency, recovery time, p50/p95 latency, latency variance, and planner prediction error.]

[Earlier material in this topic; Origin PDF p. 52: • consistency of distributed output relative to local inference;]

Authorization evaluation should exercise both directions of service permission, not only successful decryption. Negative cases include an unpermitted caller, an unpermitted Provider, a validly signed message with the wrong protected value, and reuse after request state is closed. Grant experiments measure how an enrolled entity learns and applies a new permission. Withdrawal experiments measure update propagation, status validation, parameter/DKEY refresh, and rejection after the update is applied. Disconnected nodes and old ciphertext are separate cases with the limitations stated in Section 3.4.1.

The collaboration ablations retain request-ID isolation and signature validation. Removing assigned-producer checks tests a wrong-role object within the same request; removing edge checks tests an undeclared input from an otherwise authorized participant. Replacement tests inject old work after reassignment. Outcomes record the check responsible for rejection and whether prohibited execution occurred. If another retained check rejects the object, that experiment does not establish the removed check's independent necessity. Valid-input controls prevent rejection of all work from being mistaken for correct enforcement.

Framework reuse can be assessed by documenting which invocation, authorization, object-transfer, and lifecycle components are shared by UAV and DI, and which adapters remain application-specific. Source size alone is not a sufficient maintainability metric. Record the changes needed to add a service type and the contracts that must be supplied. This supports the framework's engineering rationale while leaving protocol correctness and performance to their own evidence. [Old; Origin PDF p. 54: The final evaluation should connect the framework thesis to both application domains and show how the applications validate and extend NDNSF mechanisms. Table 9

summarizes the planned evidence.]

[Earlier material in this topic; Origin PDF p. 46, 52, 54: This chapter supports RQ4 by defining UAV as the command-sensitive, real-time, mobile workload used to validate NDNSF beyond core framework tests. It also feeds back into RQ2 and RQ3 by using UAV behavior to stress Targeted invocation, provider collaboration, timeout diagnosis, and large-data references.

This chapter supports RQ5 by defining distributed inference as the artifact-heavy, dependency-driven workload used to validate NDNSF under multi-stage compute/data collaboration. It also feeds back into RQ2 by testing whether large-data references, provider roles, generated plans, and long-running selection status can remain generic rather than model-specific.

6.3 Evaluation Matrix]

For the boundary study in Table 10, both applications will exercise the same NDNSF runtime paths for wrong-producer rejection, undeclared dependencies, and late work after cancellation. Separate inputs will pass cryptographic and dependency checks but violate an application contract: an out-of-window image or a tensor with an incompatible declared model/shape. Record which layer rejects each case and verify that rejected input cannot start dependent application work. Valid controls must still complete. A correctly shaped tensor with incorrect values illustrates the remaining limit: numerical validation against a reference, not a signature, is needed. Record the adapters and runtime changes needed for each workload; duplicated checks or new application-specific branches in the runtime would indicate that the proposed division of responsibilities needs revision. These observations support a reusable contract for the tested workloads, not universal applicability to all services.

Deployment records should include dependency versions, model hashes, topology, link parameters, clocks, retry policies, cache configuration, and key-management settings without publishing secrets. Separate application failures from startup, configuration, and collection failures. A failed setup is not evidence of a protocol's inability to complete a request. These records allow the final dissertation to distinguish supported claims, useful negative results, and configurations that remain untested.

6.5 Primary Research Scope and Feasibility

The initial UAV graph will assign two image-collection roles and one analysis role; its matched control uses the same images and recognition algorithm. The initial DI case will target a two-Provider YOLO (You Only Look Once) partition, with every tensor crossing the partition boundary declared as a dependency. Model version, cut, precision, input set, and numerical tolerance will be fixed after graph-compatibility checks and before comparative runs. A feasible unsplit execution supplies the numerical reference and performance control. These are planned configurations, not completed validations.

[Earlier material in this topic; Origin PDF p. 53: Chapter 6
Research Plan and Timeline]

The minimum scope includes valid completion, invalid-input rejection, and one controlled cancellation/replacement case in each graph. Additional partition strategies, LLM streaming, larger deployments, and real-flight trials are extensions, not prerequisites for these bounded framework findings. Existing control, telemetry, video, and single-Provider inference workflows retain their validation roles. If either core graph cannot be validated, its RQ2 claim remains unsupported and the scope or schedule must be reconsidered with the committee. The analysis will report both beneficial and unfavorable operating conditions.

Validation proceeds from local correctness tests to controlled MiniNDN campaigns, UAV simulation and bench integration, and supported physical devices. Each stage establishes evidence only for the tested environment. Physical-flight experiments require the appropriate approval; simulation alone cannot establish real-flight safety or perception performance.

6.6 Existing Foundation and Remaining Research Work

The invocation protocol, application prototypes, candidate-closure and plan-commit checks, and preliminary campaigns provide the existing foundation. Remaining work comprises the complete authorization alternative, integrated dependency/recovery checks, and the matched application experiments described above. The dissertation will connect each RQ answer to its reproducible evi-

dence and state the resulting limitations.

[Old; Origin PDF p. 53: The completed work includes the NDN service framework survey, NDNSF v0.1, core security design, provider selection, adaptive admission control, and preliminary MiniNDN evaluation. NDNSF v0.1 is not outside the dissertation; it is the completed first-stage core contribution on which the rest of the dissertation is built. Work after v0.1, including UAV-APP, DistributedInference, provider collaboration, and the supporting mechanisms needed to exercise it in realistic workloads, is treated as the application-driven extension and validation agenda to be completed and evaluated. Repository support may be used as an implementation component for large objects, but it is not presented as a separate dissertation thread.]

[Earlier material in this topic; Origin PDF p. 53: 6.1 Completed Work]

Work will proceed from a tested end-to-end workflow and matched control to additional scale or model diversity where these help answer a research question. Correctness tests and reproducible measurements take priority over optional features. If a model partition or flight platform is unavailable, the evaluation scope will be adjusted and the resulting limits reported.

[Old; Origin PDF p. 53, 54: The remaining dissertation work is now primarily a stabilization, integration, and evaluation effort organized into four threads:

1. Framework extensions: finalize provider collaboration as the central NDNSF extension, and stabilize supporting mechanisms for low-latency Targeted invocation, trusted local composition, large-data references, and NAC-ABE operational support, with emphasis on correctness, documentation, and regression coverage.

2. UAV application: stabilize the GS operator workflow, service-container organization, MAVLink/PX4 integration, live video, encrypted recording, object detection, and multi-UAV patrol as an application-driven validation of NDNSF.

3. Distributed inference: complete the NativeTracer/ONNX execution path, generated service plans, reference-based artifact exchange, provider assignment, layout campaigns, and failure compensation, then freeze the evaluation scenarios.

4. Evaluation and writing: run final MiniNDN, SITL, selected real-machine experiments while converting the resulting evidence into dissertation chapters.]

[Earlier material in this topic; Origin PDF p. 53: 6.2 Planned Work]

6.7 Timeline and Decision Points

Table 15: Revised internal targets; defense is conditional on evidence and committee approval.

Period	Research milestone / decision
September 2026	Agree on three RQs and bounded claims; freeze supported application graphs and implementation/evidence baseline; resolve correctness blockers
October 2026	Run predefined correctness tests and matched comparisons; reduce optional scope if a required end-to-end test has not passed
November 2026	Freeze accepted results and provenance; write RQ answers, costs, and limitations; keep missing results visibly unverified
December 2026	Deliver an integrated dissertation draft to the advisor; prioritize analysis and revision over optional features
January 2027	Advisor/committee review, revisions, defense readiness decision, and confirmation of current university submission/notice requirements
February 2027	Target final dissertation defense after required reviews and evidence closure; allow time for substantive corrections
Spring 2027	Complete approved corrections and final submission by the applicable Graduate School deadlines; confirm exact dates separately

[Old; Origin PDF p. 56: Table 10: Proposed dissertation timeline. Period Planned Work June–July 2026 Stabilize NDN SF provider collaboration, its supporting Targeted/local/large-data mechanisms, and the UAV/DistributedInference prototypes. August 2026 Complete UAV MiniNDN and SITL experiments for command lifecycle, telemetry readiness, live video, recording, map/mission workflows, and operational documentation. September 2026 Complete DistributedInference integration tests, including ONNX graph summaries, generated policies, reference-based artifacts, dependency-driven execution, activation references, and provider-failure scenarios. October 2026 Run final MiniNDN, SITL, and selected real-machine experiments; freeze experiment data, figures, tables, and experiment artifacts. November 2026 Finish dissertation writing and revision, complete committee feedback cycles, and defend the dissertation in November 2026.]

[Earlier material in this topic; Origin PDF p. 54: 6.4 Timeline]

The February 2027 defense target depends on evidence and committee approval. CS guid-

ance calls for an advisor-approved complete dissertation to reach committee members two weeks before the defense [36]. The Graduate School lists February 5, 2027 for the graduation application and doctoral candidacy form, and March 26, 2027 for the defended final ProQuest submission [37]; these dates must be rechecked before filing. If core end-to-end evidence is missing at the January review, scope or timing will be reconsidered with the committee.

[Earlier material in this topic; Origin PDF p. 54: This chapter connects the research questions to the dissertation execution plan. It shows that each RQ has a corresponding evidence path, while leaving room for application-driven findings from UAV and DistributedInference to refine the final NDNsf design.]

6.8 Risks, Mitigation, and Limits

[Old; Origin PDF p. 54: 6.5 Risks and Mitigation]

ABE adds authority, computation, and rekeying costs; disconnected revocation has propagation limits. Candidate ACKs do not prove future capacity or honest reporting. Cache availability does not guarantee task completion. Tests confined to one process do not establish distributed failure behavior. UAV safety and perception accuracy need evidence beyond service-message delivery. Findings from selected hardware and models must be interpreted within those configurations.

Table 16: Risks, detection evidence, and scope-preserving responses.

Risk	How it becomes visible	Response
Dependency or deployment mismatch	A binary, model, credential, or runtime configuration differs across hosts	Freeze versions and artifact identities; verify the deployment before measuring a request chain
Integration delays	End-to-end trace fails before a validated terminal result	Freeze one supported graph; separate provisioning, protocol, and application failures before adding features
Communication overhead outweighs execution benefits	Boundary transfer and setup offset any parallel-execution benefit	Report the crossover or negative result; do not hide it by changing precision or baselines
Delayed permission withdrawal	Disconnected participants retain stale status or old keys	Measure propagation and rekeying; enforce declared status validity; exclude retroactive secrecy claims
Unavailable hardware or flight approval	Required device, operating condition or approval cannot be obtained	Use declared simulation/bench scope; retain real-hardware questions as limitations
Unreproducible or confounded results	Missing provenance, unequal candidates, or inconsistent failure denominators	Preserve records, repair controls and repeat only the affected claim; withhold unsupported conclusions

[Old; Origin PDF p. 54: • Dependency risk: UAV and AI applications depend on many libraries. Mitigation: controlled dependency environments and container-based tests.

• Real flight risk: physical UAV testing is safety-critical. Mitigation: start with MiniNDN and SITL, then use controlled real-machine tests without propellers before flight.

• Performance risk: security and sync overhead may affect latency. Mitigation: use Targeted invocation, local invocation, and large-data references as supporting mechanisms where the collaborative path would otherwise add avoidable overhead, together with sampled logging and careful benchmark profiles.

• Scope risk: UAV and AI inference are both large applications. Mitigation: implement minimal convincing prototypes first, then extend only the features needed to demonstrate the framework.]

Chapter 7

Conclusion

This dissertation studies a reusable service framework over NDN. It builds on NDNSF's initial ability to discover multiple candidates and select one Provider for an invocation, evaluates its invocation and data-exchange mechanisms, and extends them to complementary Provider roles and named dependencies.

[Old; Origin PDF p. 57: This proposal presents NDNSF as a data-centric service framework for dynamic, secure, and collaborative networked applications. NDNSF v0.1 demonstrates that NDN can support a generic service abstraction with authorization, provider selection, and adaptive behavior. The proposed work centers the next framework step on provider collaboration, supported by Targeted invocation, trusted local composition, and large-data references, and validates this direction through two demanding application domains: UAV networks and distributed AI inference.

The expected outcome is not merely another application implementation, but a framework and design methodology for building service-oriented systems where services, data, permissions, and collaboration plans are named and verifiable. UAV-APP and DistributedInference are important because they stress NDNSF in different ways and reveal what the framework must provide, but the dissertation contribution remains the NDNSF framework, its mechanisms, and the evidence that those mechanisms generalize beyond a single application.]

RQ1 will identify conditions favoring ABE permission reuse or complete role-authorized alternatives as permission overlap, membership, and withdrawal frequency change. RQ2 will determine how assignments made after ACK collection constrain new and explicitly reused named objects without allowing old execution messages to advance their replacements. RQ3 will measure benefits and costs across UAV and DI workflows, and test which checks the framework can share and which require application-specific validation.

These are proposed design and protocol contributions, not a new encryption primitive or the first use of named computation. Existing campaigns establish limited invocation findings; the collaboration and cross-application claims still require their declared end-to-end evidence. The proposal asks the committee to assess the research scope, comparisons, validation criteria, and

feasibility. The dissertation will report supported answers and limitations, including conditions in which the proposed mechanisms offer no advantage.

Appendix A

Preliminary Prototype Observations and Evidence Limits

This appendix records preliminary observations from prototype records that informed this proposal [38]. It is a provenance ledger, not a second results section: the observations are not newly reproduced, and the raw traces, denominators, implementation identity, workload, and measurement definitions must be recovered before a final claim relies on them.

Table 17: Preliminary prototype observations retained for traceability and their permitted interpretation.

Observation	Prototype record	Permitted use and limit
Load/admission at 100 requests/s	p95 4,254.39 ms and 3.55% timeout without admission; about 81 admitted requests/s, p95 268.12 ms, and 0% timeout with admission. Denominators and accepted/rejected work are unverified.	Motivates separate load, accepted-work, latency, and timeout reporting; does not establish equal-work improvement.
Historical ACK suppression	2,209 selections/2,250 requests and 1.82% success for an overloaded-nearest-Provider policy; 100% for a suppression variant.	Motivates a declared positive/negative ACK experiment; not evidence about the current ACK protocol or general authorization.
Two-Provider availability	74.62% completion at 10 requests/s and 73.73% at 30 requests/s; coverage intervals, deadlines, and failure causes are unrecovered.	Motivates reachability and failure-category measurements; does not establish universal mobility or retry benefit.
Unequal-provider comparisons	Archived tables compare single-Provider baselines with multi-Provider NDNFSF configurations.	Motivates the matched Provider-count and failure-schedule campaigns in Chapter 6; cannot isolate a framework effect.

[Earlier material in this topic; Origin PDF p. 36: Table 6: Selected mechanism results from the submitted NDNFSF v0.1 evaluation. Mechanism Baseline Behavior NDNFSF Behavior Adaptive admission under 100 RPS offered load Without admission control, P95 latency reached 4254.39

ms and timeout rate reached 3.55%. With admission control, admitted rate was limited to about 81 RPS, P95 latency stayed at 268.12 ms, and timeout rate was 0.00%. Selective ACK with nearest provider overloaded ACK-all selected the nearest overloaded provider for 2209 of 2250 requests and achieved 1.82% success. Selective ACK suppressed overloaded ACKs, shifted selection to available providers, and achieved 100.00% success. Real-machine intermittent availability Single-provider NSC/gRPC stayed near 50% success when the chosen provider was unavailable for half the run. NDNSF selected among two providers and reached 74.62% success at 10 RPS and 73.73% at 30 RPS, close to the expected availability window.]

bottomrule

The ledger preserves favorable and unfavorable observations without upgrading them to current evidence. The planned evaluation will either reproduce each entry under controlled conditions or cite it only as motivation for controlled work.

References

[1] L. Zhang, A. Afanasyev, J. Burke, V. Jacobson, kc claffy, P. Crowley, C. Papadopoulos, L. Wang, B. Zhang, Named data networking, in: ACM SIGCOMM Computer Communication Review, 2014, pp. 66–73.

[2] Named Data Networking Project, NDN Packet Format Specification: Data Packet, <https://docs.named-data.net/NDN-packet-spec/current/data.html>, , accessed September 9, 2026.

[3] Y. Yu, A. Afanasyev, D. Clark, k. claffy, V. Jacobson, L. Zhang, Schematizing trust in named data networking, in: Proceedings of the 2nd ACM Conference on Information-Centric Networking, 2015, pp. 177–186. doi:10.1145/2810156. 2810170.

[4] T. Li, W. Shang, A. Afanasyev, L. Wang, L. Zhang, A brief introduction to ndn dataset synchronization (ndn sync), in: 2018 IEEE Military Communications Conference, 2018, pp. 612–618.

[Old; Origin PDF p. 57: [3] T. Li, W. Shang, A. Afanasyev, L. Wang, L. Zhang, A brief introduction to ndn dataset synchronization (ndn sync), in: 2018 IEEE Military Communications Conference, 2018, pp. 612–618.]

[5] P. Moll, V. Patil, L. Wang, L. Zhang, Sok: The evolution of distributed dataset synchronization solutions in ndn, in: Proceedings of the 9th ACM Conference on Information-Centric Networking, 2022, pp. 33–44.

[Old; Origin PDF p. 57: [4] P. Moll, V. Patil, L. Wang, L. Zhang, Sok: The evolution of distributed dataset synchronization solutions in ndn, in: Proceedings of the 9th ACM Conference on Information-Centric Networking, 2022, pp. 33–44.]

[6] C. Tschudin, M. Sifalakis, Named functions and cached computations, in: 2014 IEEE 11th Consumer Communications and Networking Conference, 2014, pp. 851–857.

[Old; Origin PDF p. 58: [13] C. Tschudin, M. Sifalakis, Named functions and cached computations, in: 2014 IEEE 11th Consumer Communications and Networking Conference, 2014, pp. 851–857.]

[7] M. Krol, I. Psaras, Nfaas: Named function as a service, in: Proceedings of the 4th ACM Conference on Information-Centric Networking, 2017, pp. 134–144.

[Old; Origin PDF p. 58: [14] M. Krol, I. Psaras, Nfaas: Named function as a service, in: Proceedings of the 4th ACM Conference on Information-Centric Networking, 2017, pp. 134–144.]

[8] M. Krol, K. Habak, D. Oran, D. Kutscher, I. Psaras, Rice: Remote method invocation in icn, in: Proceedings of the 5th ACM Conference on Information-Centric Networking, 2018, pp. 1–11.

[Old; Origin PDF p. 58: [16] M. Krol, K. Habak, D. Oran, D. Kutscher, I. Psaras, Rice: Remote method invocation in icn, in: Proceedings of the 5th ACM Conference on Information-Centric Networking, 2018, pp. 1–11.]

[9] D. Meirovitch, L. Zhang, Nsc–named service calls, or a remote procedure call for ndn, Tech. Rep. NDN-0074, Revision 1, UCLA Computer Science Department (2021). URL <https://named-data.net/wp-content/uploads/2021/05/ndn-tr-0074-1-nsc.pdf>

[Earlier material in this topic; Origin PDF p. 58: [17] T. Meirovitch, L. Zhang, Nsc–named service calls, or a remote procedure call for ndn, in: NDN Technical Report NDN-0074, Revision 1, 2021, pp. 1–20.]

[10] K. Nichols, Lessons learned building a secure network measurement framework using basic ndn, in: Proceedings of the 6th ACM Conference on Information-Centric Networking, 2019, pp. 112–122. doi:10.1145/3357150.3357397.

[Old; Origin PDF p. 58: [18] K. Nichols, Lessons learned building a secure network measurement framework using basic ndn, in: Proceedings of the 6th ACM Conference on Information-Centric Networking, 2019, pp. 112–122.]

[11] C. Yi, A. Afanasyev, I. Moiseenko, L. Wang, B. Zhang, L. Zhang, A case for stateful forwarding plane, Computer Communications 36 (7) (2013) 779–791.

[Old; Origin PDF p. 57: [2] C. Yi, A. Afanasyev, I. Moiseenko, L. Wang, B. Zhang, L. Zhang, A case for stateful forwarding plane, Computer Communications 36 (7) (2013) 779–791.]

[12] ndn-cxx: Ndn c++ library with experimental extensions, <https://github.com/named-data/ndn-cxx>, , accessed 2026.

[Old; Origin PDF p. 58: [6] ndn-cxx: Ndn c++ library with experimental extensions, <https://github.com/named-data/ndn-cxx>, , accessed 2026.]

[13] Statevectorsync: A library for distributed real-time applications over ndn, <https://github.com/named-data/ndn-svs>, , accessed 2026.

[Old; Origin PDF p. 57: [5] Statevectorsync: A library for distributed real-time applications over ndn, <https://github.com/named-data/ndn-svs>, , accessed 2026.]

[Earlier material in this topic; Origin PDF p. 58: [7] Nac: Name-based access control library for named data networking, <https://github.com/named-data/name-based-access-control>, , accessed

2026.]

[14] grpc: A high performance, open source universal rpc framework, <https://grpc.io/>, , accessed 2026.

[Old; Origin PDF p. 58: [8] grpc: A high performance, open source universal rpc framework, <https://grpc.io/>, , accessed 2026.]

[15] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, A. Y. Ng, Ros: an open-source robot operating system, ICRA Workshop on Open Source Software.
[Old; Origin PDF p. 58: [10] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, A. Y. Ng, Ros: an open-source robot operating system, ICRA Workshop on Open Source Software.]

[16] Ros 2 documentation, <https://docs.ros.org/>, , accessed 2026.
[Old; Origin PDF p. 58: [11] Ros 2 documentation, <https://docs.ros.org/>, , accessed 2026.]

[17] V. John, X. Liu, A survey of distributed message broker queues, arXiv preprint arXiv:1704.00411.
[Old; Origin PDF p. 58: [12] V. John, X. Liu, A survey of distributed message broker queues, arXiv preprint arXiv:1704.00411.]

[18] M. S. ud din, M. A. ur Rehman, B.-S. Kim, Cfec: An ultra-low latency microservices-based in-network computing framework for information-centric iovs, IEEE Transactions on Services Computing 17 (6) (2024) 3199–3212.
[Old; Origin PDF p. 58: [15] M. A. ur Rehman, B.-S. Kim, Cfec: An ultra-low latency microservices-based in-network computing framework for information-centric iovs, IEEE Transactions on Services Computing 17 (6) (2024) 3199–3212.]

[19] M. Imran, et al., Mia-ndn: Microservice-centric interest aggregation in named data networking, Sensors 23 (3) (2023) 1411.

[20] R. Tourani, et al., Towards security-as-a-service in multi-access edge, in: Proceedings of the 4th ACM/IEEE Symposium on Edge Computing, 2019, pp. 358–363.

[21] M. Król, S. Mastorakis, D. Oran, D. Kutscher, Compute first networking: Distributed computing meets ICN, in: Proceedings of the 6th ACM Conference on Information-Centric Networking, 2019, pp. 67–77. doi:10.1145/3357150.3357395.

[22] C. Barrios, E. Alker, M. Kumar, CaSCON: Cache-assisted service composition orchestration leveraging NDN, in: 2025 IEEE International Conference on Smart Computing (SMART-

COMP), 2025, pp. 122–129. doi:10.1109/SMARTCOMP65954. 2025.00061.

[23] S. Dulal, T. Yu, S. Liu, A. R. Thieme, L. Zhang, L. Wang, Enhancing NAC-ABE to support access control for mHealth applications and beyond, arXiv:2311.07299 (2023). doi:10.48550/arXiv.2311.07299. URL <https://arxiv.org/abs/2311.07299>

[24] V. Goyal, O. Pandey, A. Sahai, B. Waters, Attribute-based encryption for fine-grained access control of encrypted data, in: Proceedings of the 13th ACM Conference on Computer and Communications Security, 2006, pp. 89–98.

[Old; Origin PDF p. 58: [21] V. Goyal, O. Pandey, A. Sahai, B. Waters, Attribute-based encryption for fine-grained access control of encrypted data, in: Proceedings of the 13th ACM Conference on Computer and Communications Security, 2006, pp. 89–98.]

[25] A. Koub^{aa}, A. Allouch, M. Alajlan, Y. Javed, A. Belghith, M. Khalgui, Micro air vehicle link (mavlink) in a nutshell: A survey, IEEE Access 7 (2019) 87658–87680. doi:10.1109/ACCESS.2019.2924410.

[Old; Origin PDF p. 59: [22] A. Koub^{aa}, A. Allouch, M. Alajlan, Y. Javed, A. Belghith, M. Khalgui, Micro air vehicle link (mavlink) in a nutshell: A survey, IEEE Access 7 (2019) 87658–87680. doi:10.1109/ACCESS.2019.2924410.]

[26] Px4 autopilot user guide, <https://docs.px4.io/>, , accessed 2026.
[Old; Origin PDF p. 59: [23] Px4 autopilot user guide, <https://docs.px4.io/>, , accessed 2026.]

[27] Minindn: A mininet-based ndn emulator, <https://github.com/named-data/mini-ndn>, , accessed 2026.

[28] ONNX Runtime Authors, How to configure the past present share buffer, <https://onnxruntime.ai/docs/genai/howto/past-present-share-buffer.html>, , accessed September 11, 2026.

[29] Open neural network exchange (onnx), <https://github.com/onnx/onnx>, , accessed 2026.
[Old; Origin PDF p. 59: [27] Open neural network exchange (onnx), <https://github.com/onnx/onnx>, , accessed 2026.]

[30] Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars, L. Tang, Neurosurgeon: Collaborative intelligence between the cloud and mobile edge, in: Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems, 2017, pp. 615–629. doi:10.1145/3037697.3037698.

[Old; Origin PDF p. 59: [24] Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars, L. Tang, Neurosurgeon: Collaborative intelligence between the cloud and mobile edge, in: Proceedings of the Twenty-Second International Conference on Architectural Support for Programming

Languages and Operating Systems, 2017, pp. 615–629. doi:10.1145/3037697. 3037698.]

[31] S. Teerapittayanon, B. McDanel, H. T. Kung, Distributed deep neural networks over the cloud, the edge and end devices, in: 2017 IEEE 37th International Conference on Distributed Computing Systems, 2017, pp. 328–339. doi:10.1109/ICDCS.2017. 226.

[Old; Origin PDF p. 59: [25] S. Teerapittayanon, B. McDanel, H. T. Kung, Distributed deep neural networks over the cloud, the edge and end devices, in: 2017 IEEE 37th International Conference on Distributed Computing Systems, 2017, pp. 328–339. doi:10.1109/ICDCS.2017. 226.]

[32] S. Teerapittayanon, B. McDanel, H. T. Kung, Branchynet: Fast inference via early exiting from deep neural networks, in: 2016 23rd International Conference on Pattern Recognition, 2016, pp. 2464–2469. doi:10.1109/ICPR.2016.7900006.

[Old; Origin PDF p. 59: [26] S. Teerapittayanon, B. McDanel, H. T. Kung, Branchynet: Fast inference via early exiting from deep neural networks, in: 2016 23rd International Conference on Pattern Recognition, 2016, pp. 2464–2469. doi:10.1109/ICPR.2016.7900006.]

[33] NDNSF Project, Proposal research revision: Claim and evidence audit, Companion repository document: docs/PAPER/proposal-defense/research-revision-audit.md, , september 9, 2026; includes frozen campaign paths, evidence limits, and unresolved research gates.

[34] gRPC Authors, Custom name resolution, <https://grpc.io/docs/guides/custom-name-resolution/>, , accessed September 9, 2026.

[35] Named Data Networking Project, NAC: Automating access control via named data, <https://named-data.net/publications/zhang2018nac/> (2018).

[36] University of Memphis, Department of Computer Science, Information for computer science ph.d. students, https://www.memphis.edu/cs/pdfs/forms_grad_phd_programinfo.pdf, , accessed September 9, 2026.

[37] University of Memphis Graduate School, Graduate school graduation information, https://www.memphis.edu/gradschool/current_students/graduation.php, , spring 2027 deadlines, checked September 16, 2026.

[38] T. Ma, NDNSF: A data-centric service framework for secure, collaborative, and dynamic networked applications, , source manuscript for the prototype observations, University of Memphis, Tables 5–6 and Section 5.6. Historical reported observations, not newly reproduced

measurements (Jun. 2026).

[Earlier material in this topic; Origin PDF p. 58: [9] R. T. Fielding, Architectural styles and the design of network-based software architectures, Ph.D. thesis, University of California, Irvine (2000).]